University of Minho
School of Engineering

Intelligent Systems Associate Laboratory



Dados e Aprendizagem Automática

Artificial Neural Networks:
Multilayer Perceptron

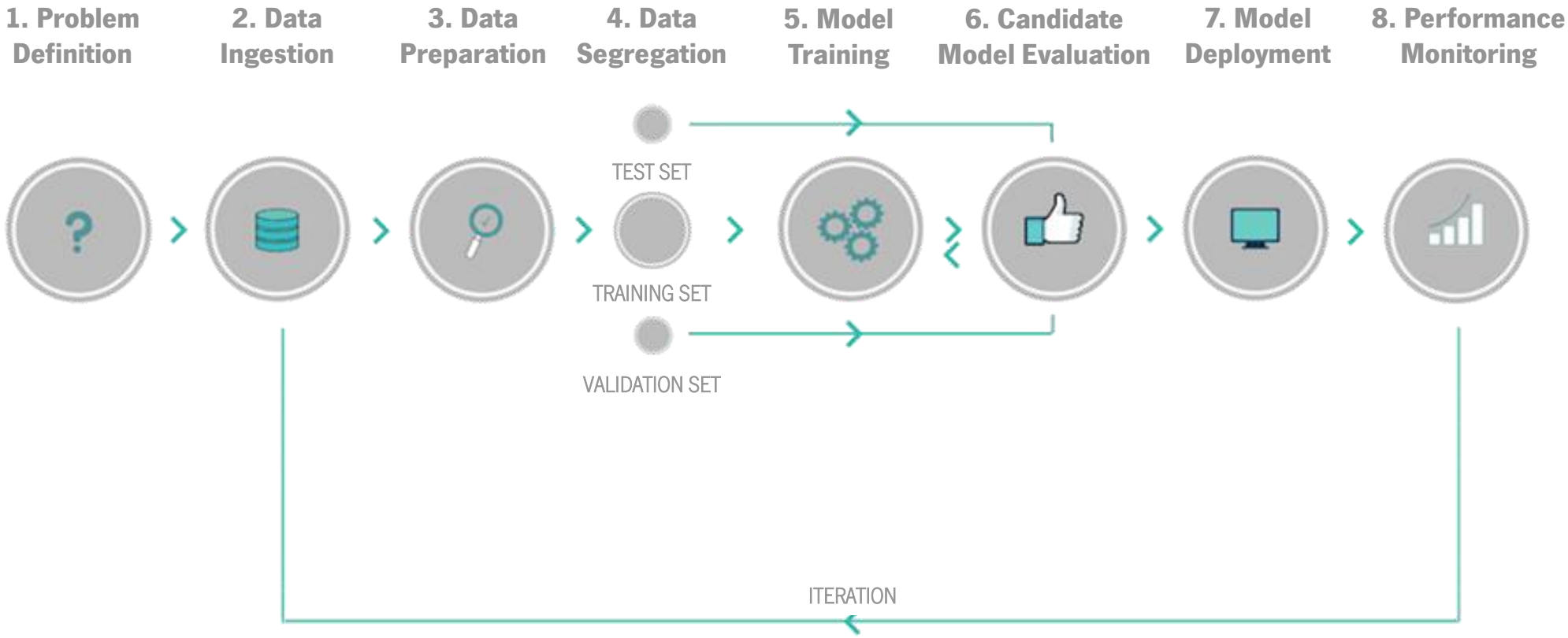
DAA 2026/2027

Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

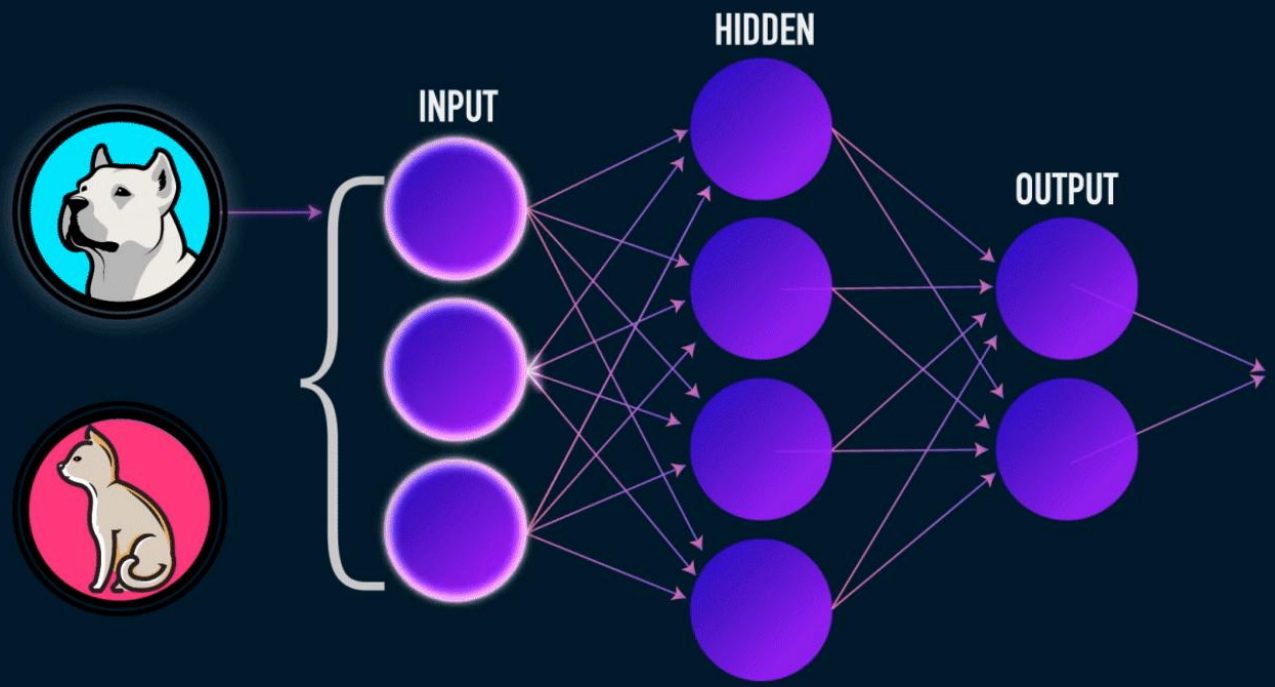
Part VII

Contents

- Artificial Neural Networks
  - Multilayer Perceptron
- Hands On

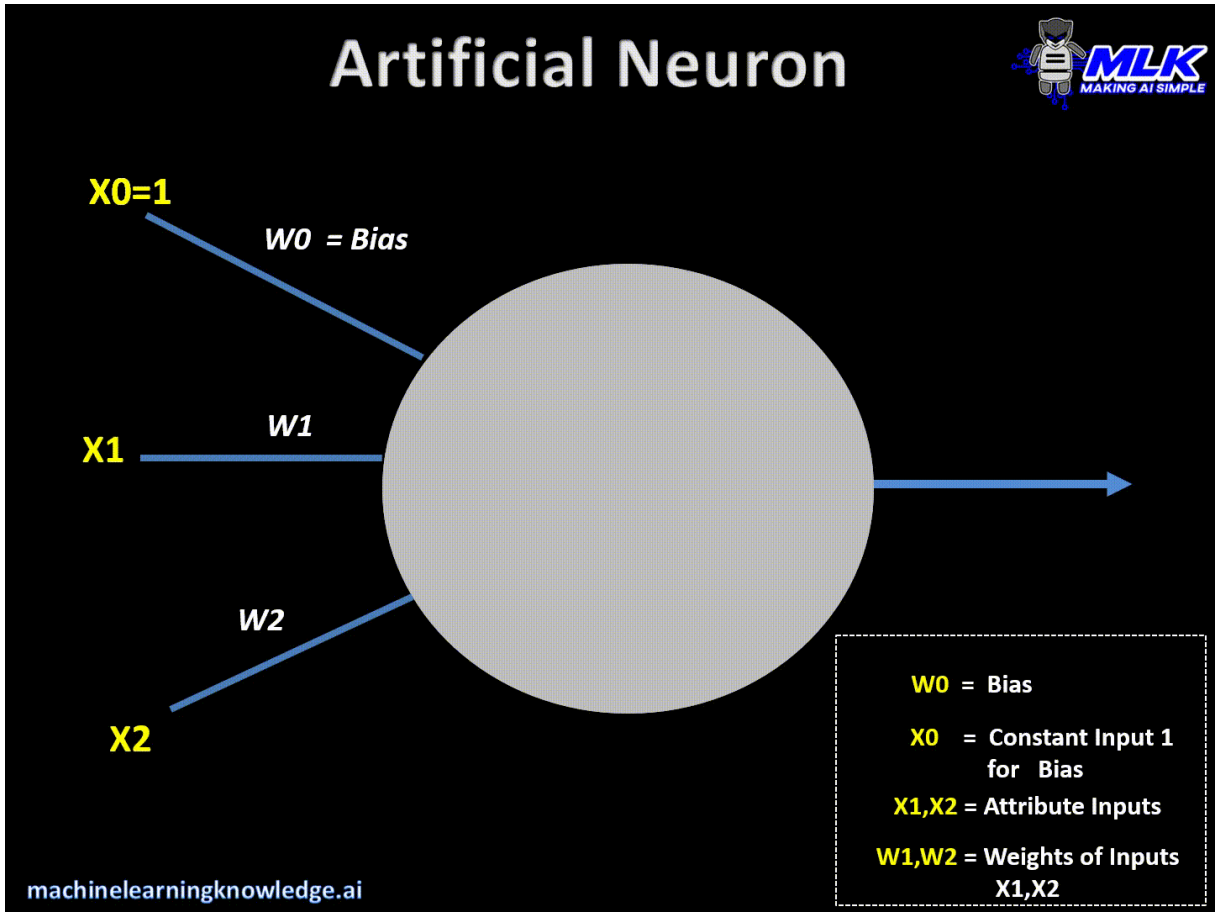

1. Problem Definition
2. Data Ingestion
3. Data Preparation
4. Data Segregation
5. Model Training
6. Candidate Model Evaluation
7. Model Deployment
8. Performance Monitoring
TEST SET
TRAINING SET
VALIDATION SET
ITERATION

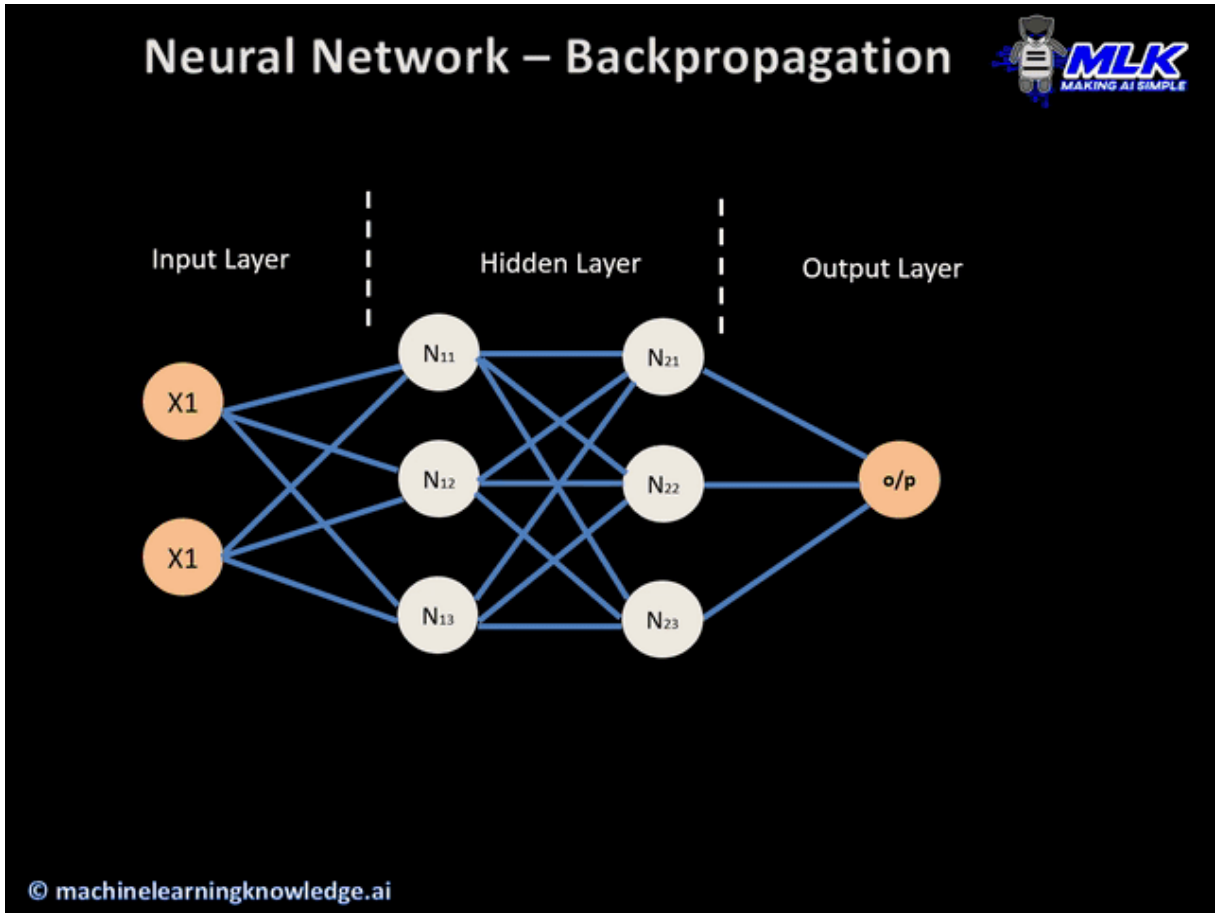
Artificial Neural Networks



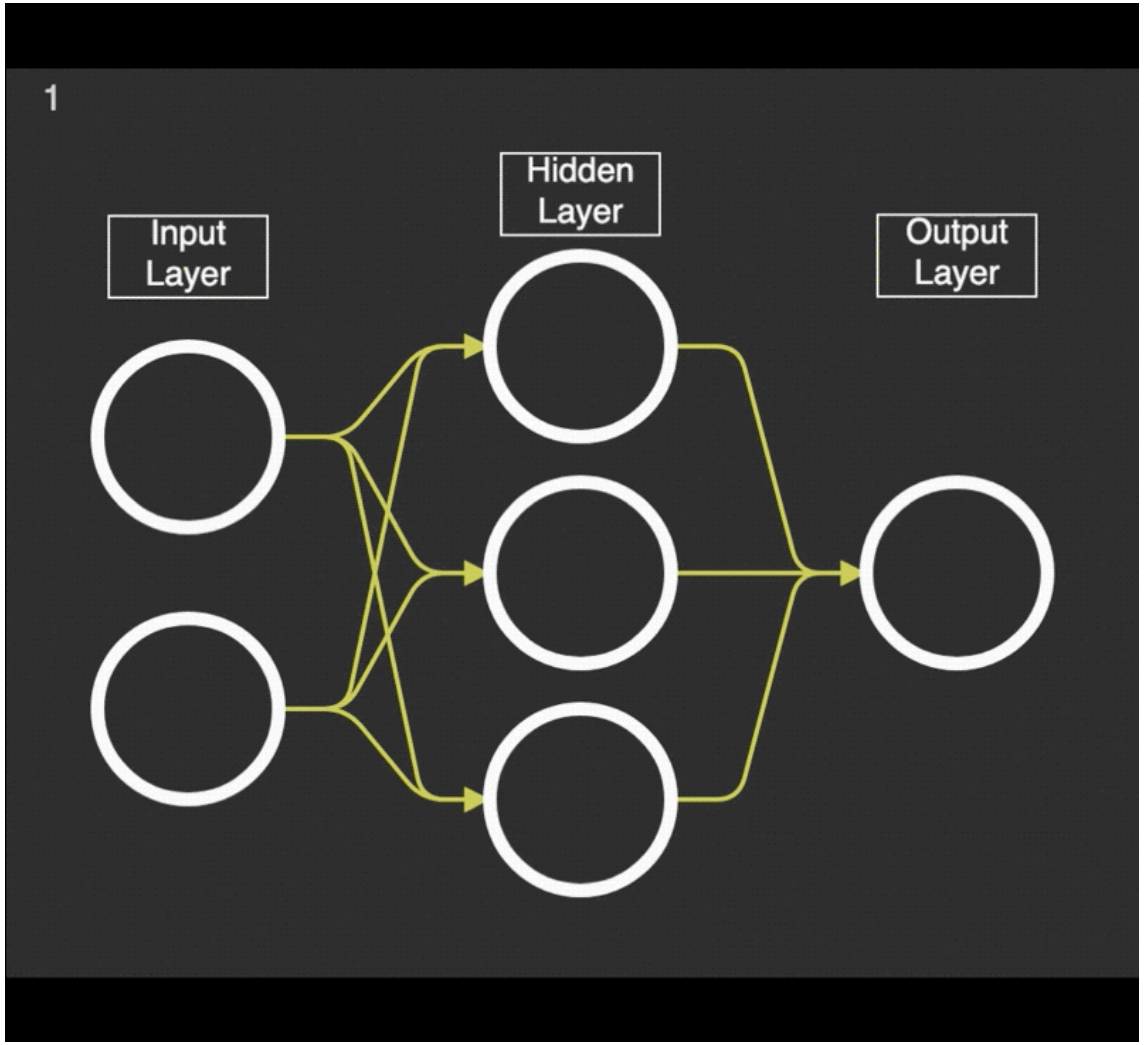
(https://miro.medium.com/max/2000/1*bhFifratH9DjKqMBTeQG5A.gif)

Artificial Neural Networks





Artificial Neural Networks



Artificial Neural Networks

Problem: development of a Machine Learning model that can **predict the class of the passengers** the embarked on the Titanic.

Model: **Multilayer Perceptrons (MLPs)**, which are a type of Artificial Neural Network.

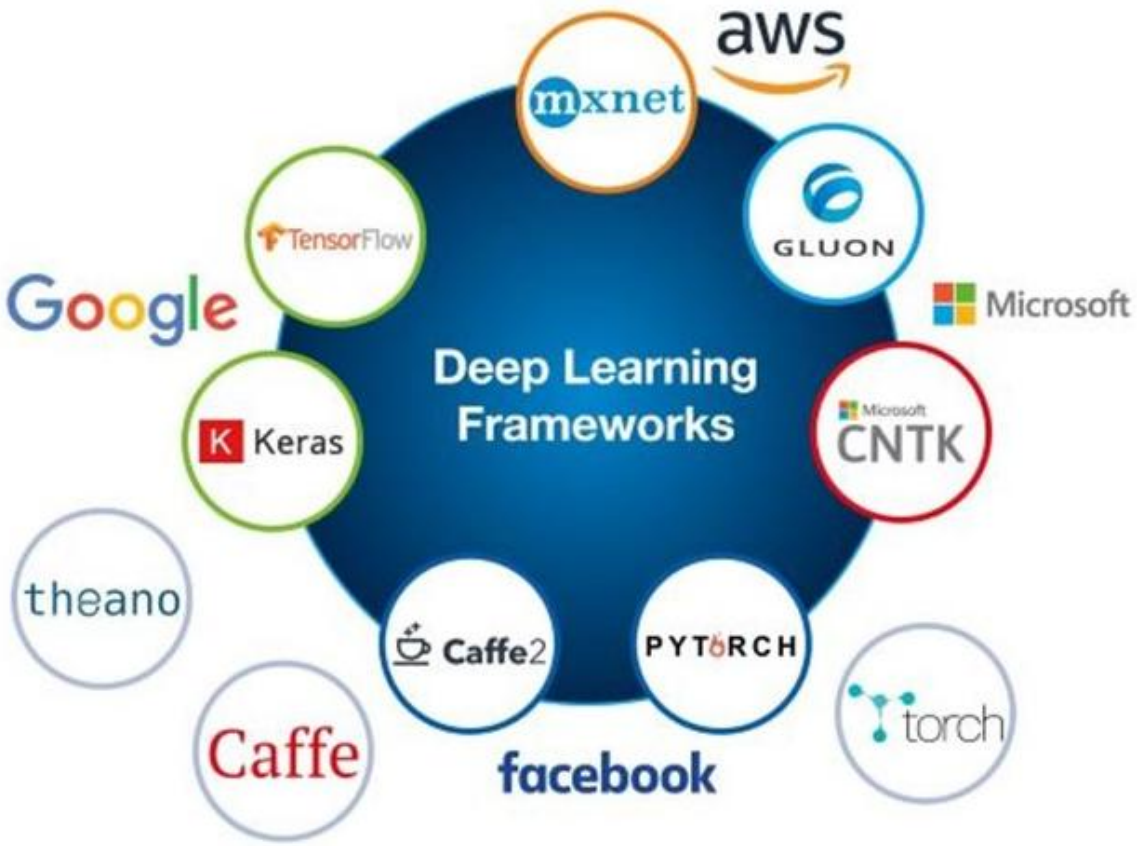
To implement our first Artificial Neural Network, we will use:



ML learning library based on the Torch library used for applications such as computer vision and natural language processing. It was originally developed by Meta AI and is now part of the Linux Foundation.

Common Frameworks

Theano	A Python library that allows to define, optimize and calculate mathematical expressions with multidimensional arrays. It works with GPUs and performs differential calculations efficiently. Developed by the University of Montreal's MLA lab
Lasagne	A light library for building and training neural networks with Theano
Blocks	A Theano-based framework for building and training neural networks
TensorFlow	Open-source library for numerical computation using graphs, developed by the Google Brain team
Keras	A deep learning library for Python that runs on either Theano or TensorFlow
MXNet	Amazon's deep learning framework is designed for efficiency and flexibility
PyTorch	An optimized and flexible library of tensors and neural networks with strong GPU support. Developed by Facebook Artificial Intelligence Research team (FAIR)
Torch	Developed by Ronan Collobert
Caffe	Developed by Berkeley Vision and Learning Center
CNTK	Developed by Microsoft
Deeplearning4j	Developed by Skymind



PyTorch



PyTorch is an **optimized tensor library for deep learning** that uses both GPUs and CPUs:

- It is an open-source software library for high-performance numerical computation with strong support for machine learning (ML) and deep learning (DL)
- Its flexibility, ease of use, performance optimization capabilities and thriving ML community have made it a top choice in the research community, as have its dynamic computational graphs, Pythonic nature and ease of use for prototyping models. It is used by many large companies, such as Amazon, Tesla, Meta and OpenAI, to power ML and AI research initiatives
- It is commonly used in applications such as image recognition and language processing and allows quicker prototyping than TensorFlow. In contrast, TensorFlow treats the NN as a static object, meaning that if you want to change the behaviour of your model, you have to start from scratch.

You may need to install some libraries.

For `pytorch`, select the configuration that suits your machine: https://pytorch.org/get-started/locally/

```
pip install livelossplot
pip install torchinfo
```

The Problem and the Data

Dataset: information regarding the **passengers** with 891 entries and 12 features, including:

- *PassengerId*
- *Survived*
- *Pclass*
- *Name*
- *Sex*
- *Age*
- *SibSp*
- *Parch*
- *Ticket*
- *Fare*
- *Cabin*
- *Embarked*

Let's use skip FE and EDA and use the dataset prepared in the last class:

```
t = pd.read_csv("titanic_ds.csv")
```

Data Split

The target is *Pclass*.
Define X and y and save it into a file.

```
df_X = t.drop('Pclass', axis=1)
df_X.head()
```

	PassengerId	Survived	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	1	22.0	1	0	7.2500	2
1	2	1	0	38.0	1	0	71.2833	0
2	3	1	0	26.0	0	0	7.9250	2
3	4	1	0	35.0	1	0	53.1000	2
4	5	0	1	35.0	0	0	8.0500	2

```
t_X = pd.DataFrame(df_X)
filename = "titanic_X.csv"
t_X.to_csv(filename, index=False, encoding='utf-8')
```

```
df_y = t['Pclass']
df_y.head()
```

```
0    3
1    1
2    3
3    1
4    3
Name: Pclass, dtype: int64
```

```
t_y = pd.DataFrame(df_y)
filename = "titanic_y.csv"
t_y.to_csv(filename, index=False, encoding='utf-8')
```

Data Preparation

For data preparation, use `torch` library to handle *data loaders* and *tensors.* Assign 70% for train data, 10% for test data and 20% for validation data.

```
def prepare_data(n_test, n_val):
    #create an instance of the dataset
    dataset = CSVDataset()
    #calculate the split
    train, test, val = dataset.get_splits(n_test, n_val)
    #prepare the data loaders
    train_dl = DataLoader(train, batch_size=len(train), shuffle=True)
    test_dl = DataLoader(test, batch_size=len(test), shuffle=True)
    val_dl = DataLoader(val, batch_size=len(val), shuffle=True)
    return train_dl, test_dl, val_dl
```

```
train_dl, test_dl, val_dl = prepare_data(0.1, 0.2)
```

```
(891, 8)
torch.Size([891])
2
1
float32
torch.int64
```

```
class CSVDataset():
    def __init__(self):
        #define inputs and outputs
        df_X = pd.read_csv("titanic_X.csv", header=0)
        df_y = pd.read_csv("titanic_y.csv", header=0)
        #convert to numpy array
        self.X = df_X.values
        self.y = df_y.values[:, 0]-1
        #ensure X and y are float32 and long tensor
        self.X = self.X.astype('float32')
        self.y = torch.tensor(self.y, dtype=torch.long, device=device)
        print(self.X.shape)
        print(self.y.shape)
        print(self.X.ndim)
        print(self.y.ndim)
        print(self.X.dtype)
        print(self.y.dtype)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        #get an instance
        return [self.X[idx], self.y[idx]]

    def get_splits(self, n_test, n_val):
        #define test and train size
        test_size = round(n_test * len(self.X))
        val_size = round(n_val * len(self.X))
        train_size = len(self.X) - (test_size + val_size)
        #return holdout split
        return random_split(self, [train_size, test_size, val_size])
```

Data Balance

Check data balance.

```
def visualize_dataset(train_dl, test_dl, val_dl):
    print(f"Train size:{len(train_dl.dataset)}")
    print(f"Test size:{len(test_dl.dataset)}")
    print(f"Validation size:{len(val_dl.dataset)}")
    x, y = next(iter(train_dl)) #iterate through the loaders to fetch a batch of cases
    print(f"Shape tensor train data batch - input: {x.shape}, output: {y.shape}")
    x, y = next(iter(test_dl))
    print(f"Shape tensor test data batch - input: {x.shape}, output: {y.shape}")
    x, y = next(iter(val_dl))
    print(f"Shape tensor validation data batch - input: {x.shape}, output: {y.shape}")
```

```
visualize_dataset(train_dl, test_dl, val_dl)
```

```
Train size:624
Test size:89
Validation size:178
Shape tensor train data batch - input: torch.Size([624, 8]), output: torch.Size([624])
Shape tensor test data batch - input: torch.Size([89, 8]), output: torch.Size([89])
Shape tensor validation data batch - input: torch.Size([178, 8]), output: torch.Size([178])
```

Data Balance

15

```
def visualize_holdout_balance(train_dl, test_dl, val_dl):
    _, y_train = next(iter(train_dl))
    _, y_test = next(iter(test_dl))
    _, y_val = next(iter(val_dl))

    sns.set_style('whitegrid')
    train_df = len(y_train)
    test_df = len(y_test)
    val_df = len(y_val)
    Class_1_train = np.count_nonzero(y_train == 0)
    Class_2_train = np.count_nonzero(y_train == 1)
    Class_3_train = np.count_nonzero(y_train == 2)
    print("train data: ", train_df)
    print("Class 1: ", Class_1_train)
    print("Class 2: ", Class_2_train)
    print("Class 3: ", Class_3_train)
    print("Values' mean (train): ", np.mean(y_train.numpy()))

    Class_1_test = np.count_nonzero(y_test == 0)
    Class_2_test = np.count_nonzero(y_test == 1)
    Class_3_test = np.count_nonzero(y_test == 2)
    print("test data: ", test_df)
    print("Class 1: ", Class_1_test)
    print("Class 2: ", Class_2_test)
    print("Class 3: ", Class_3_test)
    print("Values' mean (test): ", np.mean(y_test.numpy()))
```

```
    Class_1_test = np.count_nonzero(y_test == 0)
    Class_2_test = np.count_nonzero(y_test == 1)
    Class_3_test = np.count_nonzero(y_test == 2)
    print("test data: ", test_df)
    print("Class 1: ", Class_1_test)
    print("Class 2: ", Class_2_test)
    print("Class 3: ", Class_3_test)
    print("Values' mean (test): ", np.mean(y_test.numpy()))

    Class_1_val = np.count_nonzero(y_val == 0)
    Class_2_val = np.count_nonzero(y_val == 1)
    Class_3_val = np.count_nonzero(y_val == 2)
    print("validation data: ", val_df)
    print("Class 1: ", Class_1_val)
    print("Class 2: ", Class_2_val)
    print("Class 3: ", Class_3_val)
    print("Values' mean (val): ", np.mean(y_val.numpy()))

    graph = sns.barplot(x=['Class 1 train', 'Class 2 train', 'Class 3 train',
                           'Class 1 test', 'Class 2 test', 'Class 3 test',
                           'Class 1 val', 'Class 2 val', 'Class 3 val'],
                        y=[Class_1_train, Class_2_train, Class_3_train,
                           Class_1_test, Class_2_test, Class_3_test,
                           Class_1_val, Class_2_val, Class_3_val])

    graph.set_title('Data balance by class')
    plt.xticks(rotation=70)
    plt.tight_layout()
    plt.savefig('data_balance_MLP.png')
    plt.show()
```

Data Balance

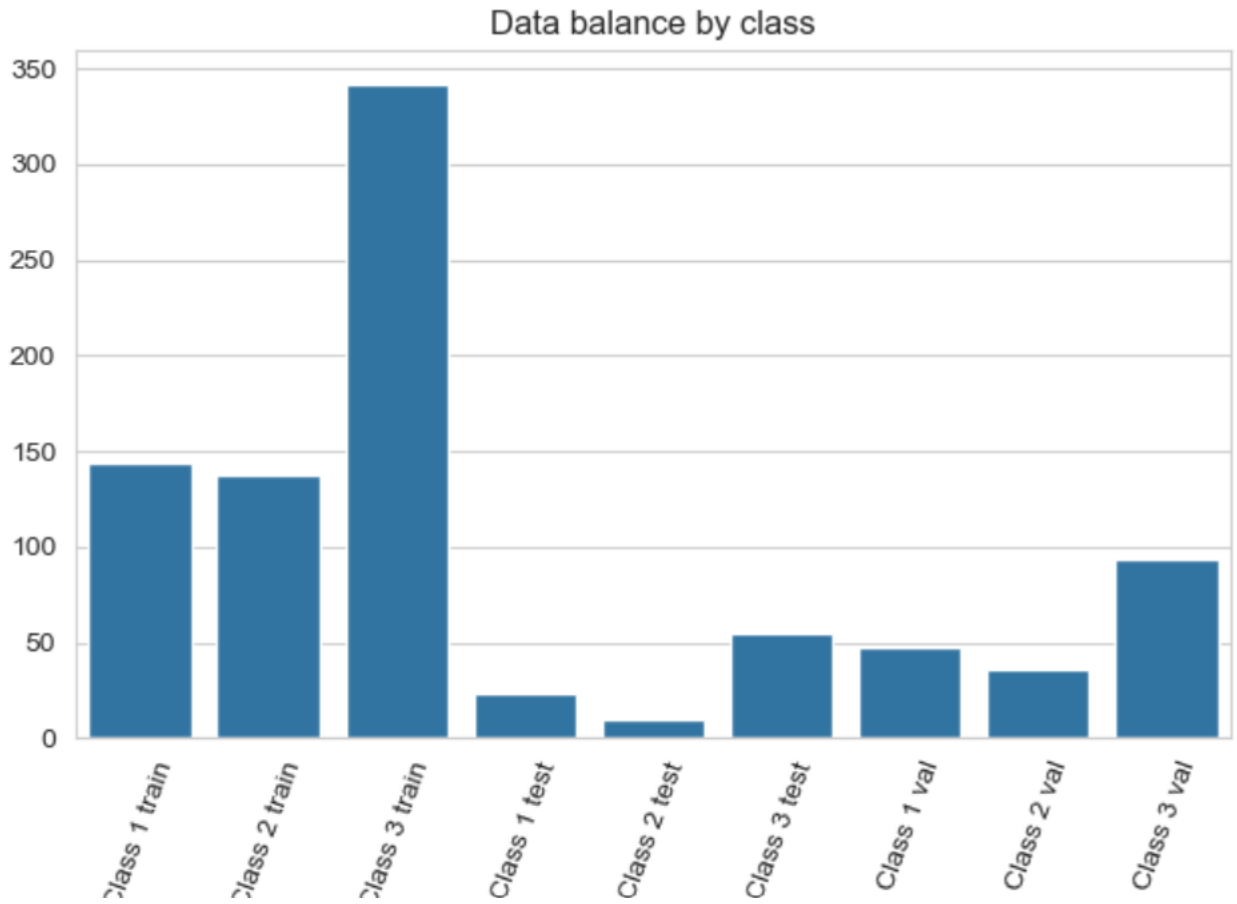

```
    graph.set_title('Data balance by class')
    plt.xticks(rotation=70)
    plt.tight_layout()
    plt.savefig('data_balance_MLP.png')
    plt.show()

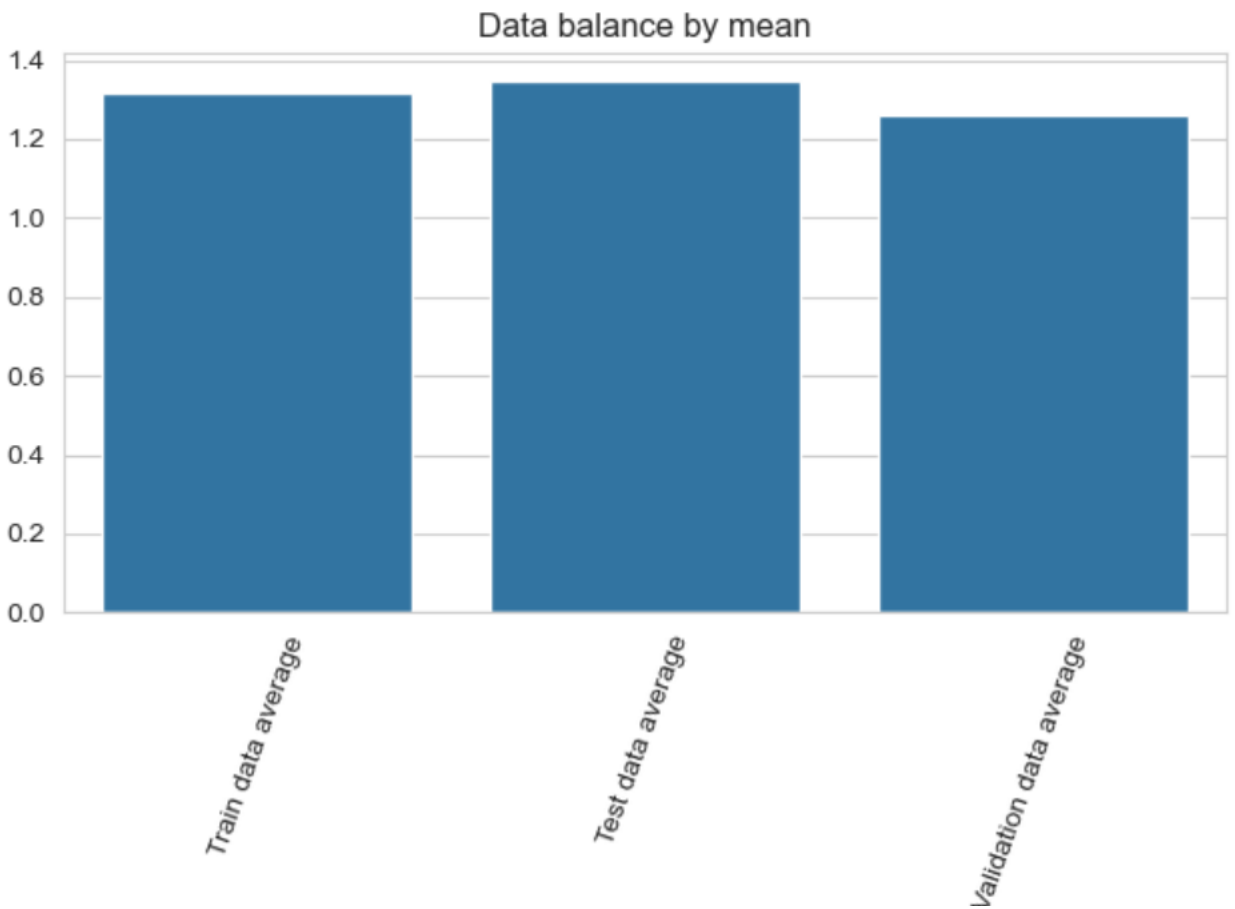
    graph = sns.barplot(x=['Train data average','Test data average', 'Validation data average'],
                        y=[np.mean(y_train.numpy()), np.mean(y_test.numpy()), np.mean(y_val.numpy())])
    graph.set_title('Data balance by mean')
    plt.xticks(rotation=70)
    plt.tight_layout()
    plt.show()
```

```
visualize_holdout_balance(train_dl, test_dl, val_dl)
```

```
train data:  624
Class 1:  144
Class 2:  138
Class 3:  342
Values' mean (train):  1.3173076923076923
test data:  89
Class 1:  24
Class 2:  10
Class 3:  55
Values' mean (test):  1.348314606741573
validation data:  178
Class 1:  48
Class 2:  36
Class 3:  94
Values' mean (val):  1.2584269662921348
```

Data Balance





Multilayer Perceptron

Build the model using the following `torch` libraries:

- `torch.nn.Module()` – the *base class* for building neural networks
- `torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)` – a *Linear* transformer
- `torch.nn.ReLU(inplace=False)` – a *Rectified Linear Unit* function for element-wise processing
- `torch.nn.Softmax(dim=None)` – a *SoftMax* function for an n-dimensional input tensor. This calculates the probability of each class.

Functions to **initialise** neural network parameters:

- `torch.nn.init.xavier_uniform_(tensor, gain=1.0, generator=None)` – fills the input tensor with values using a *Xavier Uniform Distribution*
- `torch.nn.init.kaiming_uniform_(tensor, a=0, mode='fan_in', nonlinearity='leaky_relu', generator=None)` – fills the input Tensor with values using a *Kaiming Uniform Distribution.*

Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_1(Module):
    def __init__(self, n_inputs):
        super(MLP_1, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3) #one node for the predicted value output
        #activation functions
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1) # softmax since it is multiclass
        #The dim argument is required unless your input tensor is a vector
        #if we do not put an activation function at the end, we consider a linear activation

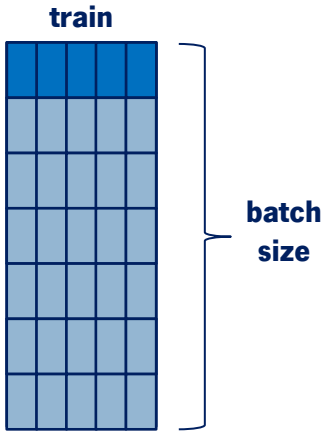
    #network architecture
    def forward(self, X):
        #input for the 1s layer
        X = self.hidden1(X)
        X = self.act1(X)
        #2nd layer
        X = self.hidden2(X)
        X = self.act1(X)
        #3rd layer and output
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

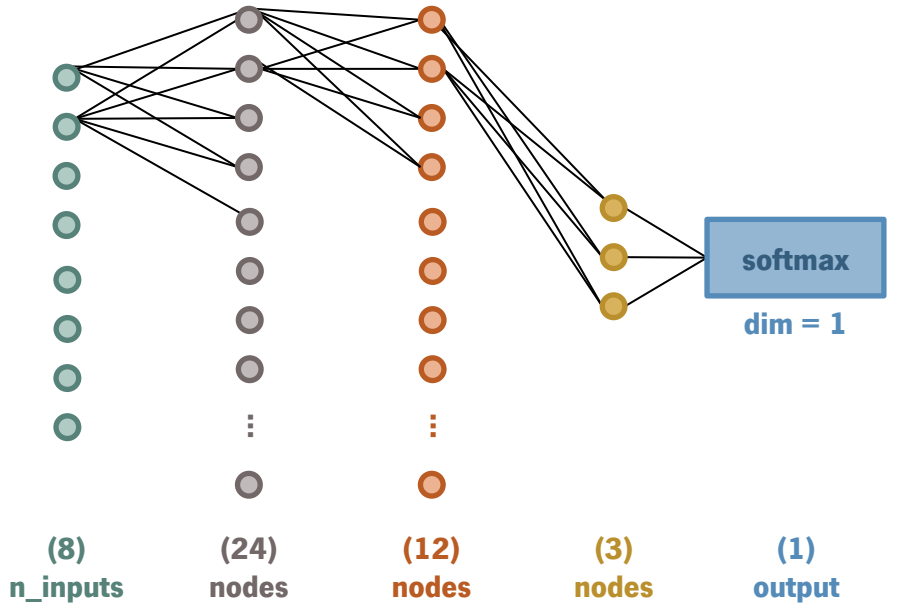
```
model = MLP_1(8)
```

Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD





Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

number of times each case is trained by the net

```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_1(Module):
    def __init__(self, n_inputs):
        super(MLP_1, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3) #one node for the predicted value output
        #activation functions
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1) # softmax since it is multiclass
        #The dim argument is required unless your input tensor is a vector
        #if we do not put an activation function at the end, we consider a linear activation

    #network architecture
    def forward(self, X):
        #input for the 1s layer
        X = self.hidden1(X)
        X = self.act1(X)
        #2nd layer
        X = self.hidden2(X)
        X = self.act1(X)
        #3rd layer and output
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

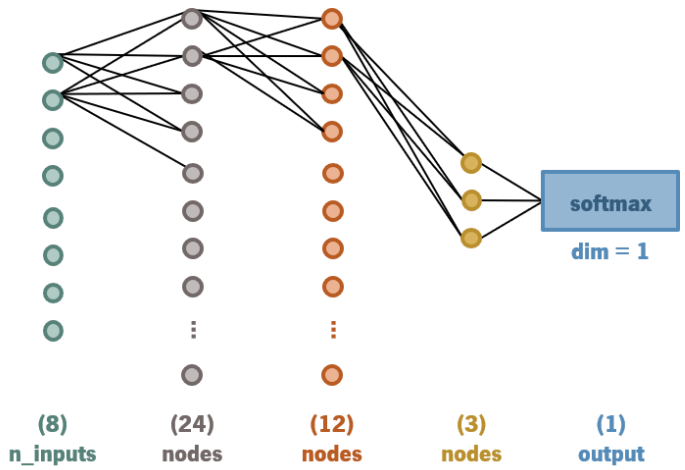
```
model = MLP_1(8)
```

Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

number of input features




```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_1(Module):
    def __init__(self, n_inputs):
        super(MLP_1, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3) #one node for the predicted value output
        #activation functions
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1) # softmax since it is multiclass
        #The dim argument is required unless your input tensor is a vector
        #if we do not put an activation function at the end, we consider a linear activation

    #network architecture
    def forward(self, X):
        #input for the 1s layer
        X = self.hidden1(X)
        X = self.act1(X)
        #2nd layer
        X = self.hidden2(X)
        X = self.act1(X)
        #3rd layer and output
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

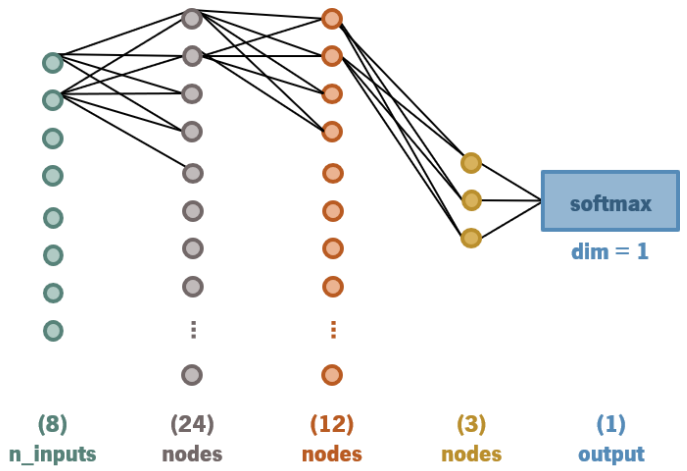
```
model = MLP_1(8)
```

Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

number of classes to predict




```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_1(Module):
    def __init__(self, n_inputs):
        super(MLP_1, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3) #one node for the predicted value output
        #activation functions
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1) # softmax since it is multiclass
        #The dim argument is required unless your input tensor is a vector
        #if we do not put an activation function at the end, we consider a linear activation

    #network architecture
    def forward(self, X):
        #input for the 1s layer
        X = self.hidden1(X)
        X = self.act1(X)
        #2nd layer
        X = self.hidden2(X)
        X = self.act1(X)
        #3rd layer and output
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

```
model = MLP_1(8)
```

Multilayer Perceptron – Model 1

Model 1

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

The `dim` argument is required unless the input tensor is a vector
If an activation function is not applied at the end, it is considered a *linear activation*

```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_1(Module):
    def __init__(self, n_inputs):
        super(MLP_1, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3) #one node for the predicted value output
        #activation functions
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1) # softmax since it is multiclass
        #The dim argument is required unless your input tensor is a vector
        #if we do not put an activation function at the end, we consider a linear activation

    #network architecture
    def forward(self, X):
        #input for the 1s layer
        X = self.hidden1(X)
        X = self.act1(X)
        #2nd layer
        X = self.hidden2(X)
        X = self.act1(X)
        #3rd layer and output
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

```
model = MLP_1(8)
```

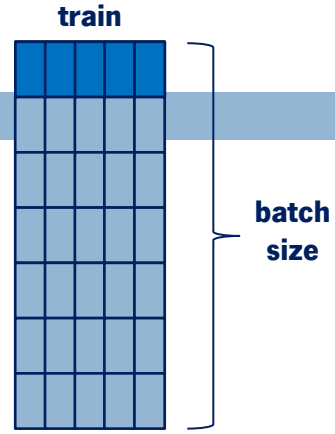
Multilayer Perceptron – Model 1

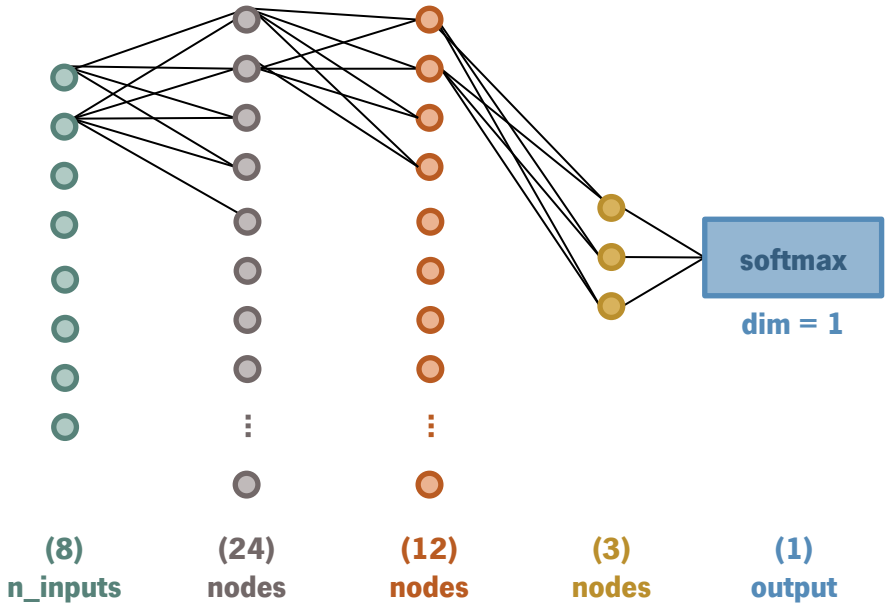
25

```
print(summary(model, input_size=(len(train_dl.dataset), 8), verbose=0))
model.to(device)
```

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
MLP_1                                    [624, 3]                  --
├─Linear: 1-1                            [624, 24]                 216
├─ReLU: 1-2                              [624, 24]                 --
├─Linear: 1-3                            [624, 12]                 300
├─ReLU: 1-4                              [624, 12]                 --
├─Linear: 1-5                            [624, 3]                  39
├─Softmax: 1-6                           [624, 3]                  --
==========================================================================================
Total params: 555
Trainable params: 555
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.35
==========================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.19
Params size (MB): 0.00
Estimated Total Size (MB): 0.22
==========================================================================================

MLP_1(
  (hidden1): Linear(in_features=8, out_features=24, bias=True)
  (hidden2): Linear(in_features=24, out_features=12, bias=True)
  (hidden3): Linear(in_features=12, out_features=3, bias=True)
  (act1): ReLU()
  (act2): Softmax(dim=1)
)
```





Multilayer Perceptron

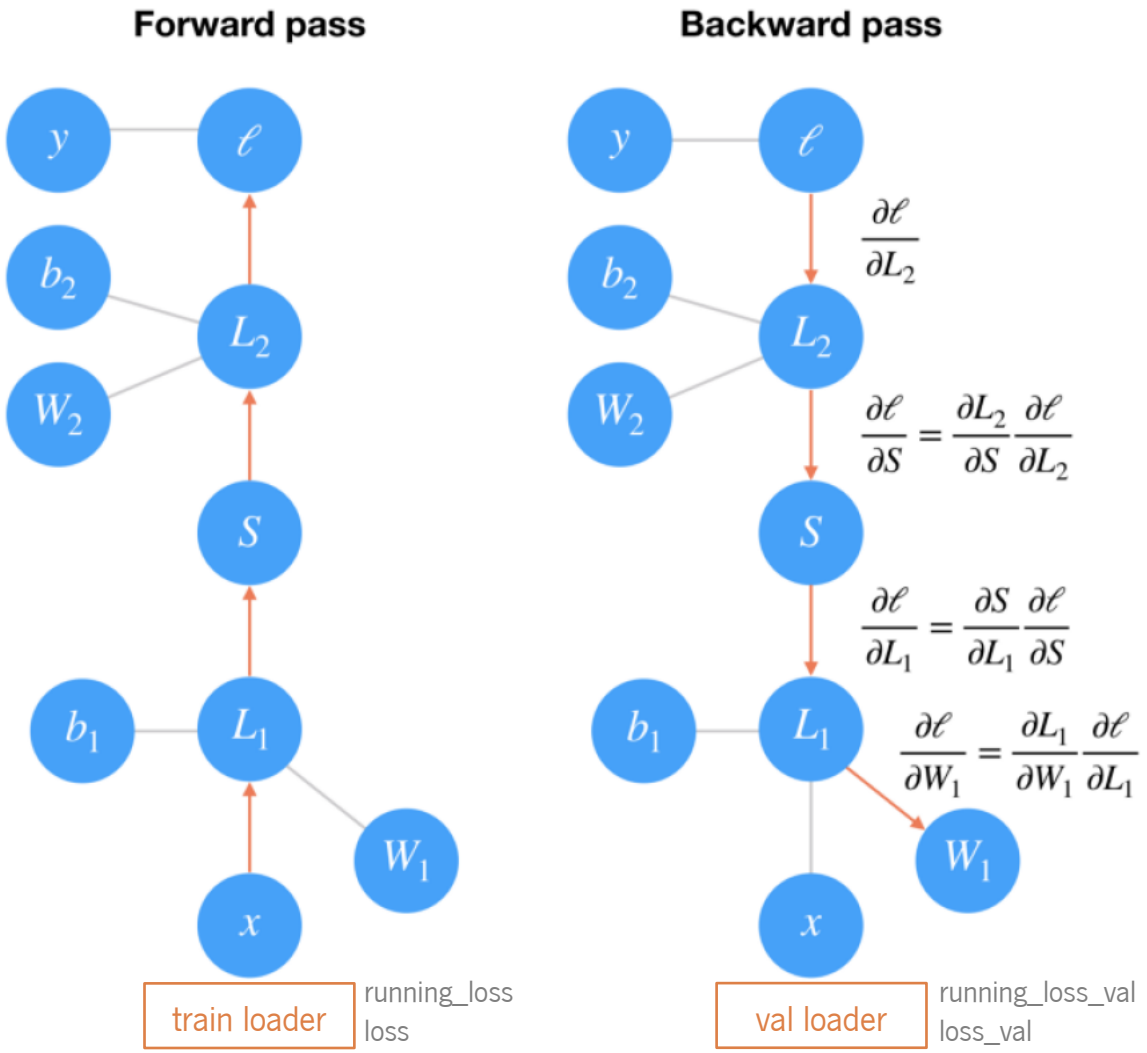
To train the model use the following `torch` libraries:

- `torch.nn.CrossEntropyLoss(weight=None, size_average=None, ignore_index=-100, reduce=None, reduction='mean', label_smoothing=0.0)` – computes the *Cross Entropy Loss* function. This is useful for training *classification problems* with C classes, as it accepts *ground truth labels* directly as integers in the range `[0, no. of classes]`. There is *no need to one-hot encode* the labels. This is particularly useful when you have an *unbalanced training set*
- `torch.optim.SGD(params, lr=0.001, momentum=0, dampening=0, weight_decay=0, nesterov=False, *, maximize=False, foreach=None, differentiable=False, fused=None)` – implements *Stochastic Gradient Descent* as an optimiser
- `torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0, amsgrad=False, *, foreach=None, maximize=False, capturable=False, differentiable=False, fused=None)` – implements the *Adam* algorithm as an optimizer.

Multilayer Perceptron – Model 1

Train the model.

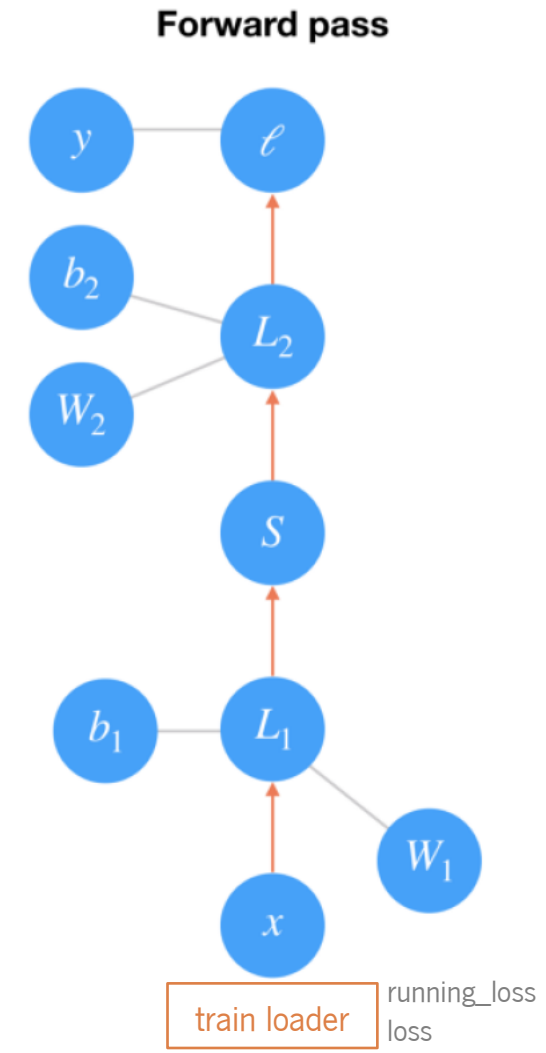
```
def train_model(train_dl, val_dl, model):
    #to visualize the training process
    liveloss = PlotLosses()
    #define loss function and optimization
    criterion = CrossEntropyLoss() #sparse_categorical_crossentropy
    optimizer = SGD(model.parameters(), lr=LEARNING_RATE, momentum=0.9)
```



Multilayer Perceptron – Model 1

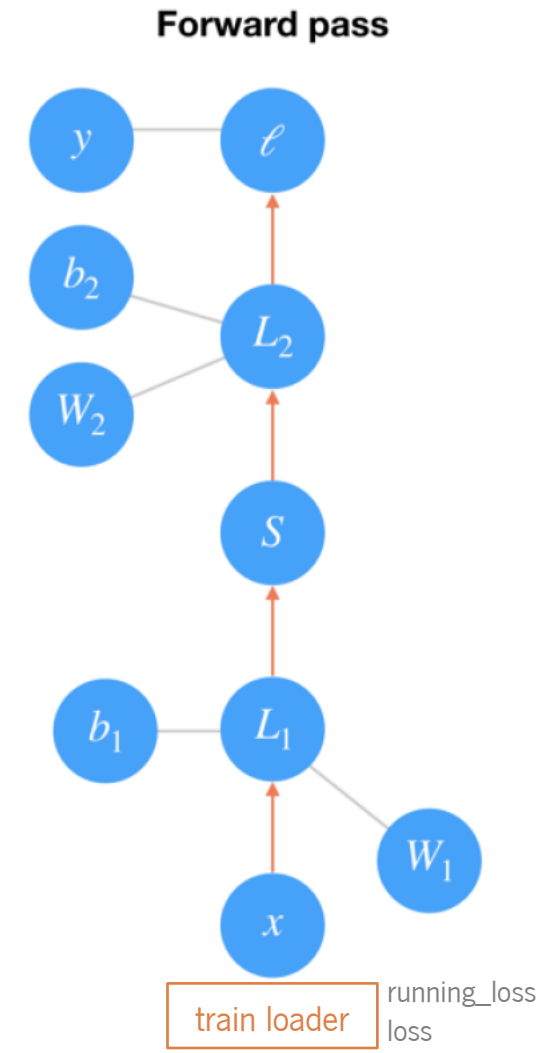
Train the model.

```
def train_model(train_dl, val_dl, model):
    #to visualize the training process
    liveloss = PlotLosses()
    #define loss function and optimization
    criterion = CrossEntropyLoss() #sparse_categorical_crossentropy
    optimizer = SGD(model.parameters(), lr=LEARNING_RATE, momentum=0.9)
    #iterate the epochs
    for epoch in range(EPOCHS):
        logs = {} #
        #train phase
        model.train()
        running_loss  = 0.0
        running_corrects  = 0.0
        #backpropagation
        for inputs, labels in train_dl:
            inputs = inputs.to(device)
            labels = labels.to(device)
            #calculate model output
            outputs = model(inputs)
            #calculate the loss
            loss = criterion(outputs, labels)
```



Multilayer Perceptron – Model 1


```
#iterate the epochs
for epoch in range(EPOCHS):
    logs = {} #
    #train phase
    model.train()
    running_loss  = 0.0
    running_corrects  = 0.0
    #backpropagation
    for inputs, labels in train_dl:
        inputs = inputs.to(device)
        labels = labels.to(device)
        #calculate model output
        outputs = model(inputs)
        #calculate the loss
        loss = criterion(outputs, labels)
        optimizer.zero_grad() #sets the gradients of all parameters to zero
        loss.backward()
        #update model weights
        optimizer.step()
        running_loss += loss.detach() * inputs.size(0)
        _, preds = torch.max(outputs, 1) # Get predictions from the maximum value
        running_corrects += torch.sum(preds == labels.data)
    epoch_loss = running_loss / len(train_dl.dataset)
    epoch_acc = running_corrects.float() / len(train_dl.dataset)
    logs['loss'] = epoch_loss.item()
    logs['accuracy'] = epoch_acc.item()
```



Multilayer Perceptron – Model 1

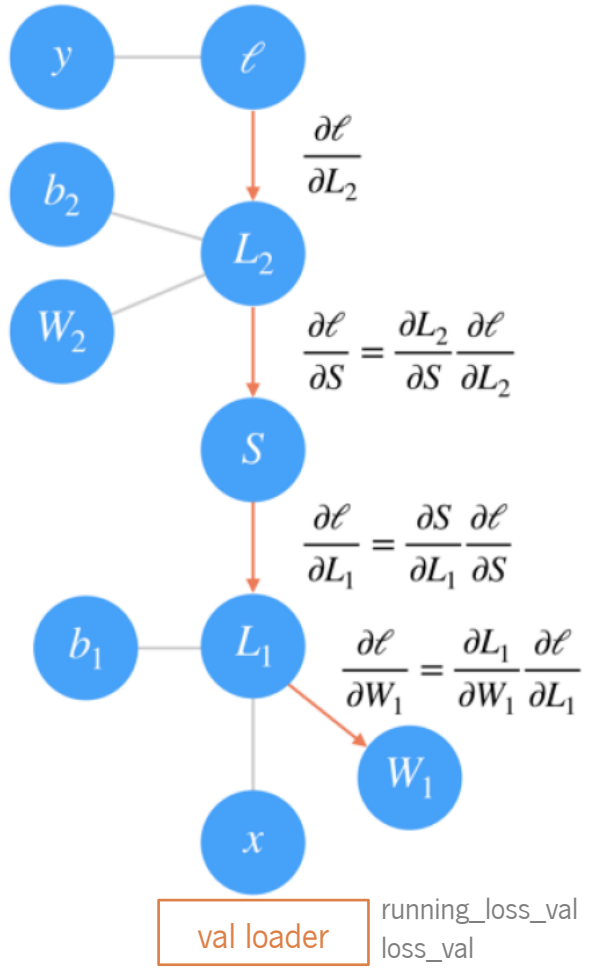

```
        logs['loss'] = epoch_loss.item()
        logs['accuracy'] = epoch_acc.item()

        #Validation phase
        model.eval()
        running_loss_val  = 0.0
        running_corrects_val  = 0.0
        for inputs, labels in val_dl:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss_val = criterion(outputs, labels)
            running_loss_val += loss_val.detach() * inputs.size(0)
            _, preds = torch.max(outputs, 1) # Get predictions from the maximum value
            running_corrects_val += torch.sum(preds == labels.data)
        epoch_loss_val = running_loss_val / len(val_dl.dataset)
        epoch_acc_val = running_corrects_val.float() / len(val_dl.dataset)
        logs['val_loss'] = epoch_loss_val.item()
        logs['val_accuracy'] = epoch_acc_val.item()
        liveloss.update(logs)
        liveloss.send()
```

```
train_model(train_dl, val_dl, model)
```

Forward pass

Backward pass



val loader

running_loss_val
loss_val

Multilayer Perceptron – Model 1


```
        logs['loss'] = epoch_loss.item()
        logs['accuracy'] = epoch_acc.item()

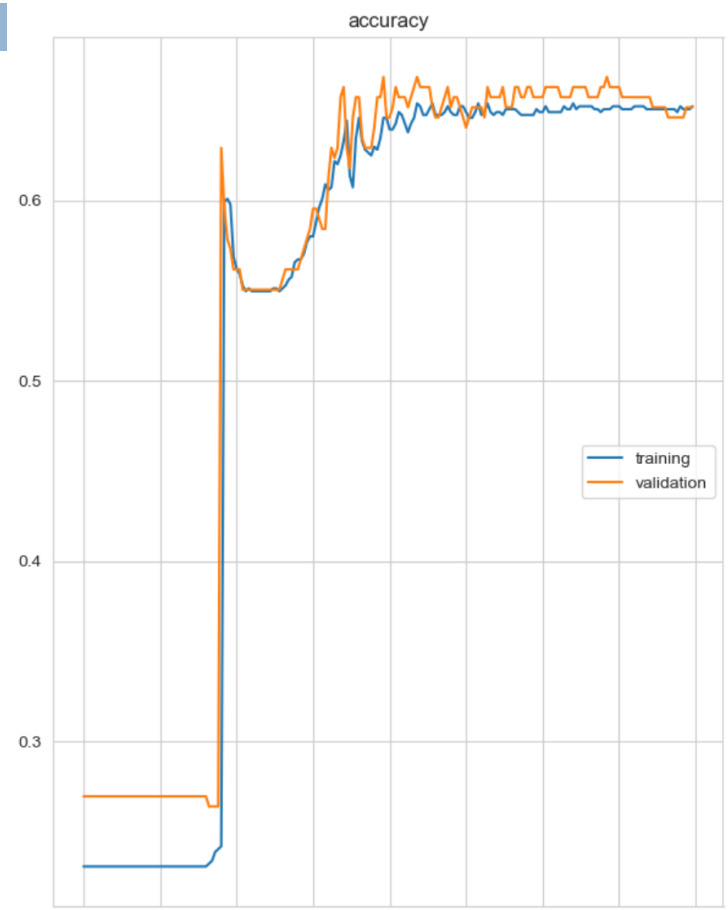
        #Validation phase
        model.eval()
        running_loss_val  = 0.0
        running_corrects_val  = 0.0
        for inputs, labels in val_dl:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss_val = criterion(outputs, labels)
            running_loss_val += loss_val.detach() * inputs.size(0)
            _, preds = torch.max(outputs, 1) # Get predictions from the maximum value
            running_corrects_val += torch.sum(preds == labels.data)
        epoch_loss_val = running_loss_val / len(val_dl.dataset)
        epoch_acc_val = running_corrects_val.float() / len(val_dl.dataset)
        logs['val_loss'] = epoch_loss_val.item()
        logs['val_accuracy'] = epoch_acc_val.item()
        liveloss.update(logs)
        liveloss.send()
```

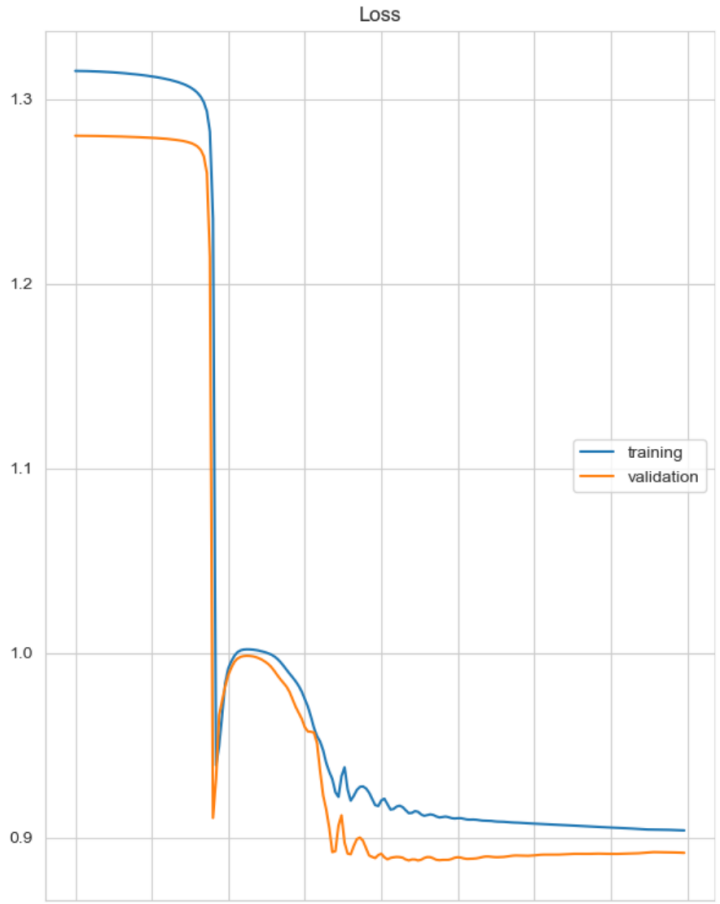
```
train_model(train_dl, val_dl, model)
```

Fraction of the data to be used as the validation set. The model will set this fraction of the data aside, will not use it for training, and will evaluate the loss and the model metrics on this data at the end of each epoch.

Multilayer Perceptron – Model 1

Plot the live training results.






```
accuracy
        training                (min:    0.231, max:    0.654, cur:    0.652)
        validation              (min:    0.264, max:    0.669, cur:    0.652)
Loss
        training                (min:    0.904, max:    1.315, cur:    0.904)
        validation              (min:    0.888, max:    1.280, cur:    0.892)
```

Multilayer Perceptron – Model 1

Evaluate the model.

```
def evaluate_model(test_dl, model):
    predictions = list()
    actual_values = list()
    for i, (inputs, labels) in enumerate(test_dl):
        #evaluate the model with test cases
        yprev = model(inputs)
        #remove numpy array
        yprev = yprev.detach().numpy()
        actual = labels.numpy()
        #convert to labels' class
        yprev = np.argmax(yprev, axis=1)
        #reshape for stacking
        actual = actual.reshape((len(actual), 1))
        yprev = yprev.reshape((len(yprev), 1))
        #save
        predictions.append(yprev)
        actual_values.append(actual)
        break
    predictions, actual_values = np.vstack(predictions), np.vstack(actual_values)
    return predictions, actual_values
```

```
def display_confusion_matrix(cm):
    plt.figure(figsize = (16,8))
    sns.heatmap(cm,annot=True,xticklabels=['Class 1', 'Class 2', 'Class 3'],
                yticklabels=['Class 1', 'Class 2', 'Class 3'],
                annot_kws={"size": 12}, fmt='g', linewidths=.5)
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.show()
```

```
predictions, actual_values = evaluate_model(test_dl, model)
```

Multilayer Perceptron – Model 1


```
success = 0
failure = 0
for r,p in zip(actual_values, predictions):
    print(f'real:{r+1} prediction:{p+1}')
    if r==p: success+=1
    else: failure+=1
```

```
real:[2] prediction:[3]
real:[2] prediction:[3]
real:[1] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[1] prediction:[3]
real:[1] prediction:[1]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[1] prediction:[3]
real:[3] prediction:[3]
real:[2] prediction:[1]
real:[1] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[1] prediction:[3]
real:[1] prediction:[1]
real:[3] prediction:[1]
```

```
acc = accuracy_score(actual_values, predictions)
print(f'Accuracy: {acc:0.3f}\n')
print(f'success:{success} failure:{failure}')
```

```
Accuracy: 0.730

success:65 failure:24
```

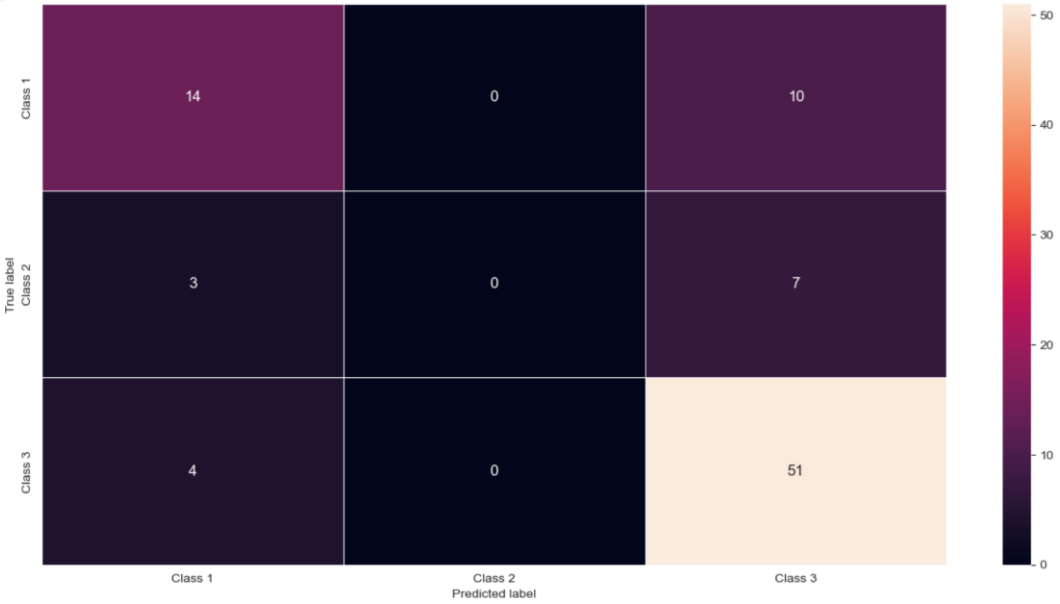
```
print(classification_report(actual_values, predictions, zero_division=0))
cm = confusion_matrix(actual_values, predictions)
print(cm)
display confusion matrix(cm)
```

```
              precision    recall  f1-score   support

           0       0.67      0.58      0.62        24
           1       0.00      0.00      0.00        10
           2       0.75      0.93      0.83        55

    accuracy                           0.73        89
   macro avg       0.47      0.50      0.48        89
weighted avg       0.64      0.73      0.68        89

[[14  0 10]
 [ 3  0  7]
 [ 4  0 51]]
```



Multilayer Perceptron – Model 1

Apply the model and make a prediction for one case.

```
def predict(row, model):
    #convert row to tensor
    row = Tensor([row])
    #make a prediction
    yprev = model(row)
    #remove the numpy array
    yprev = yprev.detach().numpy()
    return yprev
```

```
row = [5, 0, 1, 34, 0, 1, 8, 1]
yprev = predict(row, model)
print('Predicted: %s (class=%d)' % (yprev, np.argmax(yprev)+1))
```

```
Predicted: [[0.09614567 0.09562671 0.8082276 ]] (class=3)
```

What can be modified to improve this model?

Multilayer Perceptron – Model 2

Create another model and implement initializations.

Model 2:

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **initialisations**: kaiming and xavier
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

```
class MLP_2(Module):
    def __init__(self, n_inputs):
        super(MLP_2, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3)
        #initializations
        kaiming_uniform_(self.hidden1.weight, nonlinearity='relu')
        kaiming_uniform_(self.hidden2.weight, nonlinearity='relu')
        xavier_uniform_(self.hidden3.weight)
        self.act1 = ReLU()
        self.act2 = Softmax(dim=1)

    def forward(self, X):
        X = self.hidden1(X)
        X = self.act1(X)
        X = self.hidden2(X)
        X = self.act1(X)
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

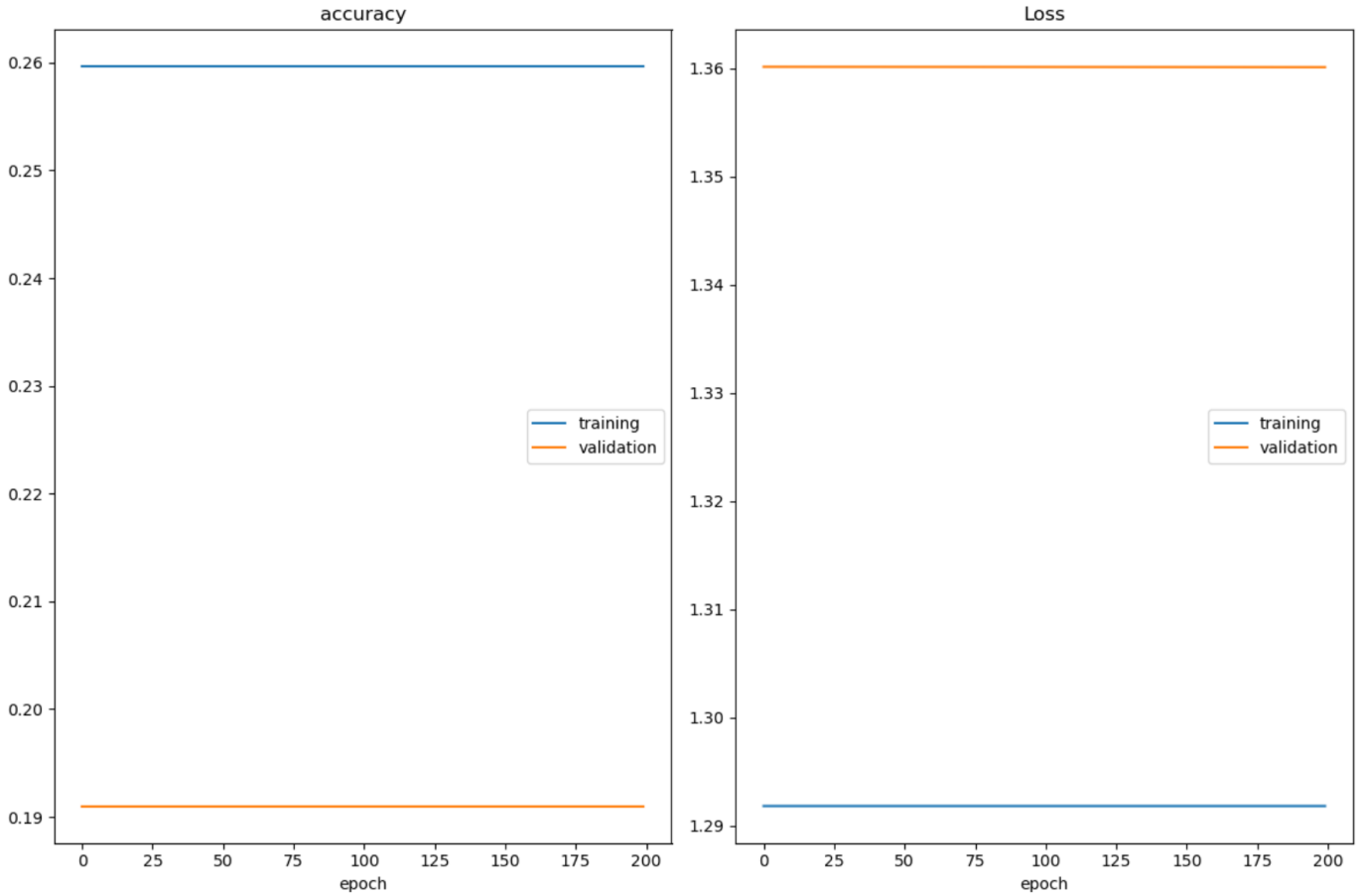
Multilayer Perceptron – Model 2


```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
MLP_1                                    [624, 3]                  --
├─Linear: 1-1                            [624, 24]                 216
├─ReLU: 1-2                              [624, 24]                 --
├─Linear: 1-3                            [624, 12]                 300
├─ReLU: 1-4                              [624, 12]                 --
├─Linear: 1-5                            [624, 3]                  39
├─Softmax: 1-6                           [624, 3]                  --
==========================================================================================
Total params: 555
Trainable params: 555
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.35
==========================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.19
Params size (MB): 0.00
Estimated Total Size (MB): 0.22
==========================================================================================

MLP_1(
  (hidden1): Linear(in_features=8, out_features=24, bias=True)
  (hidden2): Linear(in_features=24, out_features=12, bias=True)
  (hidden3): Linear(in_features=12, out_features=3, bias=True)
  (act1): ReLU()
  (act2): Softmax(dim=1)
)
```

Multilayer Perceptron – Model 2

Train the model and plot it.




```
accuracy
        training                (min:    0.260, max:    0.260, cur:    0.260)
        validation              (min:    0.191, max:    0.191, cur:    0.191)
Loss
        training                (min:    1.292, max:    1.292, cur:    1.292)
        validation              (min:    1.360, max:    1.360, cur:    1.360)
```

Multilayer Perceptron – Model 2

Evaluate the model.

```
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[1] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[1] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[2] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[3] prediction:[1]
real:[2] prediction:[1]
real:[3] prediction:[1]
```

```
Accuracy: 0.225

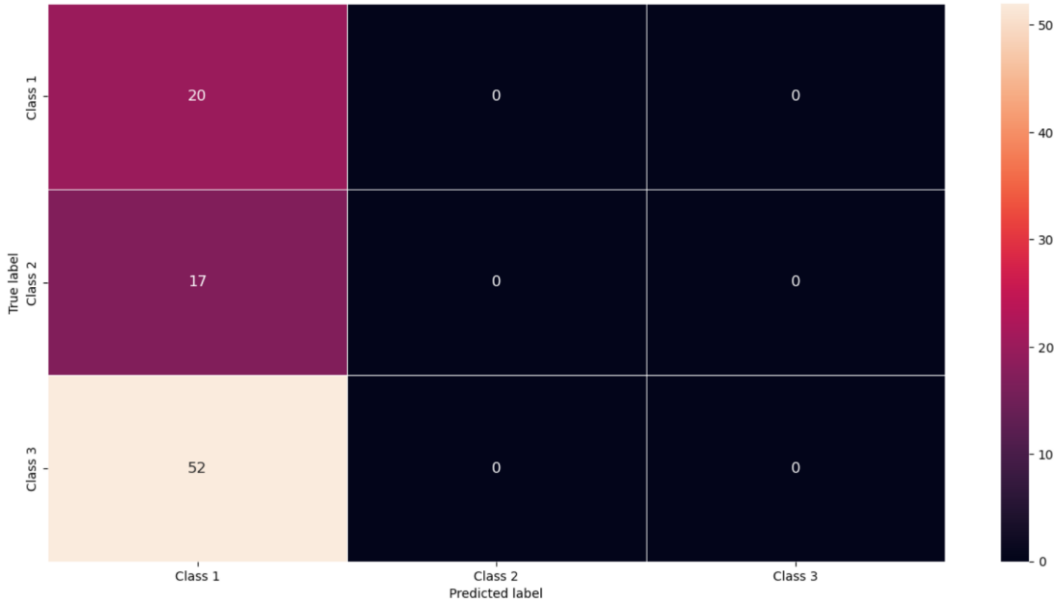
success:20 failure:69

              precision    recall  f1-score   support

           0       0.22      1.00      0.37        20
           1       0.00      0.00      0.00        17
           2       0.00      0.00      0.00        52

    accuracy                           0.22        89
   macro avg       0.07      0.33      0.12        89
weighted avg       0.05      0.22      0.08        89

[[20  0  0]
 [17  0  0]
 [52  0  0]]
```



Multilayer Perceptron – Model 2

Apply the model and make a prediction for one case.

```
Predicted: [[9.6383190e-01 3.6168091e-02 4.8151177e-11]] (class=1)
```

Did initialization improve the results? Why?

Multilayer Perceptron – Model 3

Normalize the data and create another model.

Model 3:

Data scaling or normalization is the process of creating model data in a standard format to improve training accuracy and speed. Focus on the *Age* and *Fare* features:

```
df = pd.read_csv("titanic_ds.csv")
```

```
min_age = df['Age'].min()
max_age = df['Age'].max()

df['Age'] = (df['Age'] - min_age)/(max_age - min_age)
```

```
df['Age'].describe()
```

```
count    891.000000
mean       0.363679
std        0.163605
min        0.000000
25%        0.271174
50%        0.346569
75%        0.434531
max        1.000000
Name: Age, dtype: float64
```

```
min_fare = df['Fare'].min()
max_fare = df['Fare'].max()

df['Fare'] = (df['Fare'] - min_fare)/(max_fare - min_fare)
```

```
df['Fare'].describe()
```

```
count    891.000000
mean       0.062858
std        0.096995
min        0.000000
25%        0.015440
50%        0.028213
75%        0.060508
max        1.000000
Name: Fare, dtype: float64
```

Multilayer Perceptron – Model 3


```
df.head()
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	3	1	0.271174	1	0	0.014151	2
1	2	1	1	0	0.472229	1	0	0.139136	0
2	3	1	3	0	0.321438	0	0	0.015469	2
3	4	1	1	0	0.434531	1	0	0.103644	2
4	5	0	3	1	0.434531	0	0	0.015713	2

Save the new data.

```
t = pd.DataFrame(df)
filename = "titanic_scaled.csv"
t.to_csv(filename, index=False, encoding='utf-8')
```

Multilayer Perceptron – Model 3

Redefine X and y.

```
df_X = t.drop('Pclass', axis=1)
df_X.head()
```

	PassengerId	Survived	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	1	0.271174	1	0	0.014151	2
1	2	1	0	0.472229	1	0	0.139136	0
2	3	1	0	0.321438	0	0	0.015469	2
3	4	1	0	0.434531	1	0	0.103644	2
4	5	0	1	0.434531	0	0	0.015713	2

```
t_X = pd.DataFrame(df_X)
filename = "titanic_X_scaled.csv"
t_X.to_csv(filename, index=False, encoding='utf-8')
```

```
df_y = t['Pclass']
df_y.head()
```

```
0    3
1    1
2    3
3    1
4    3
Name: Pclass, dtype: int64
```

```
t_y = pd.DataFrame(df_y)
filename = "titanic_y_scaled.csv"
t_y.to_csv(filename, index=False, encoding='utf-8')
```

Multilayer Perceptron – Model 3

Prepare the data.

```
class CSVDataset():
    def __init__(self):

        df_X = pd.read_csv("titanic_X_scaled.csv", header=0)
        df_y = pd.read_csv("titanic_y_scaled.csv", header=0)

        self.X = df_X.values
        self.y = df_y.values[:, 0]-1

        self.X = self.X.astype('float32')
        self.y = torch.tensor(self.y, dtype=torch.long, device=device)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return [self.X[idx], self.y[idx]]

    def get_splits(self, n_test, n_val):
        test_size = round(n_test * len(self.X))
        val_size = round(n_val * len(self.X))
        train_size = len(self.X) - (test_size + val_size)
        return random_split(self, [train_size, test_size, val_size])
```

```
def prepare_data(n_test, n_val):
    dataset = CSVDataset()
    train, test, val = dataset.get_splits(n_test, n_val)
    train_dl = DataLoader(train, batch_size=len(train), shuffle=True)
    test_dl = DataLoader(test, batch_size=len(test), shuffle=True)
    val_dl = DataLoader(val, batch_size=len(val), shuffle=True)
    return train_dl, test_dl, val_dl
```

```
train_dl, test_dl, val_dl = prepare_data(0.1, 0.2)
```

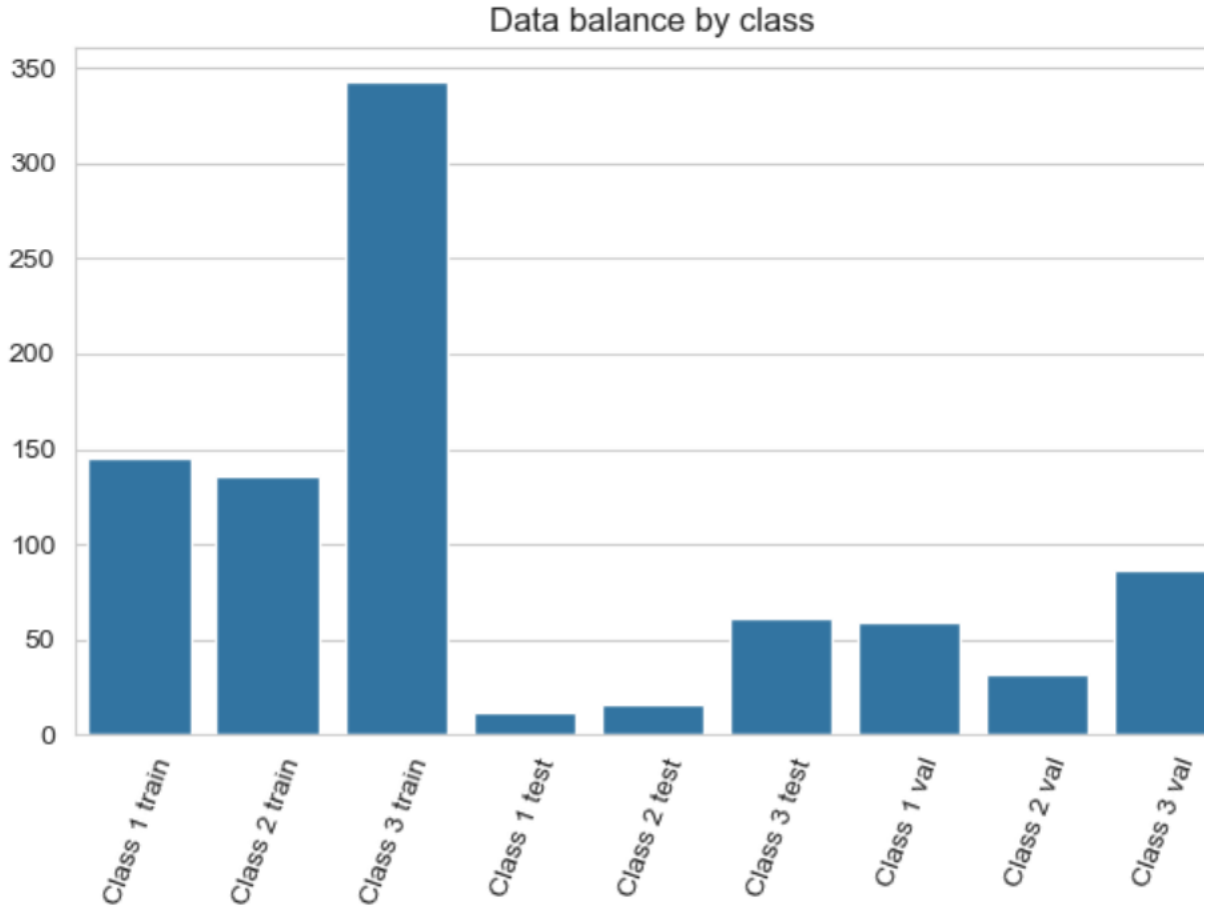
Multilayer Perceptron – Model 3

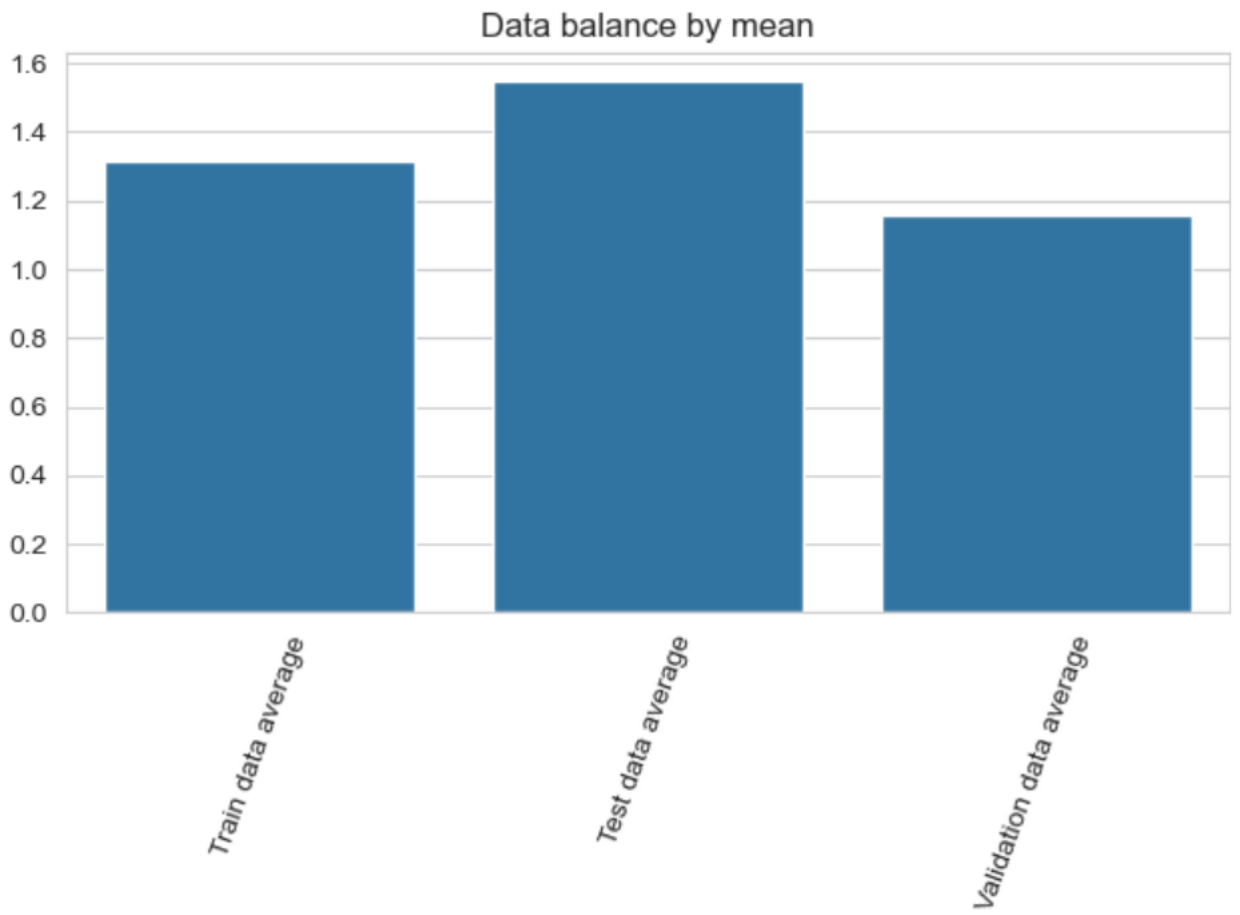
Check data balance.

```
Train size:624
Test size:89
Validation size:178
Shape tensor train data batch - input: torch.Size([624, 8]), output: torch.Size([624])
Shape tensor test data batch - input: torch.Size([89, 8]), output: torch.Size([89])
Shape tensor validation data batch - input: torch.Size([178, 8]), output: torch.Size([178])


train data:  624
Class 1:  145
Class 2:  136
Class 3:  343
Values' mean (train):  1.3173076923076923
test data:  89
Class 1:  12
Class 2:  16
Class 3:  61
Values' mean (test):  1.550561797752809
validation data:  178
Class 1:  59
Class 2:  32
Class 3:  87
Values' mean (val):  1.1573033707865168
```

Multilayer Perceptron – Model 3





Multilayer Perceptron – Model 3

Model 3

- data normalisation
- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: SGD

```
EPOCHS = 200
LEARNING_RATE = 0.001
```

```
class MLP_3(Module):
    def __init__(self, n_inputs):
        super(MLP_3, self).__init__()
        self.hidden1 = Linear(n_inputs, 24)
        self.hidden2 = Linear(24, 12)
        self.hidden3 = Linear(12, 3)

        self.act1 = ReLU()
        self.act2 = Softmax(dim=1)

    def forward(self, X):
        X = self.hidden1(X)
        X = self.act1(X)
        X = self.hidden2(X)
        X = self.act1(X)
        X = self.hidden3(X)
        X = self.act2(X)
        return X
```

```
model = MLP_3(8)
```

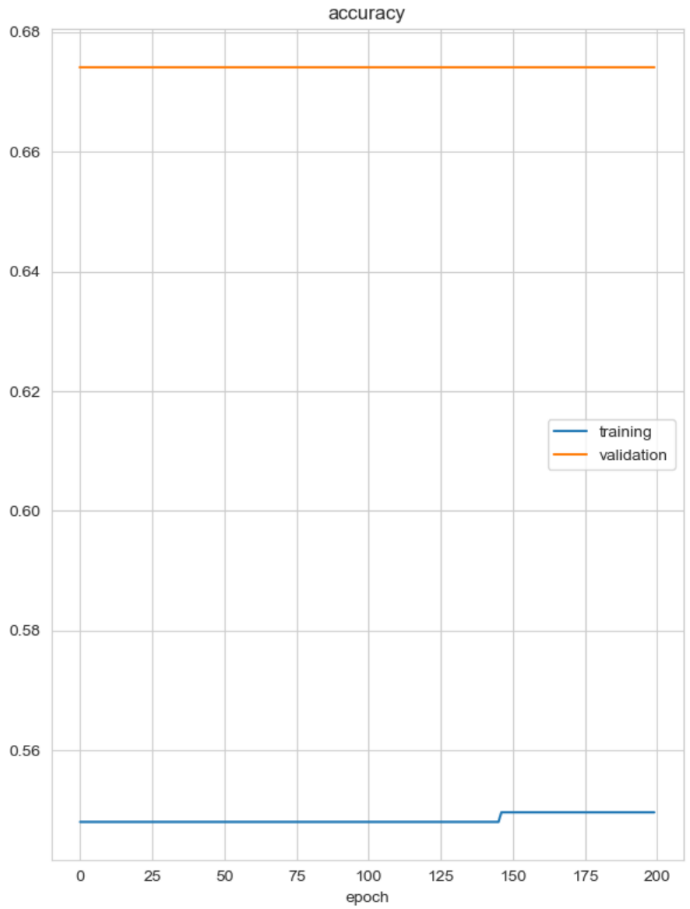
Multilayer Perceptron – Model 3

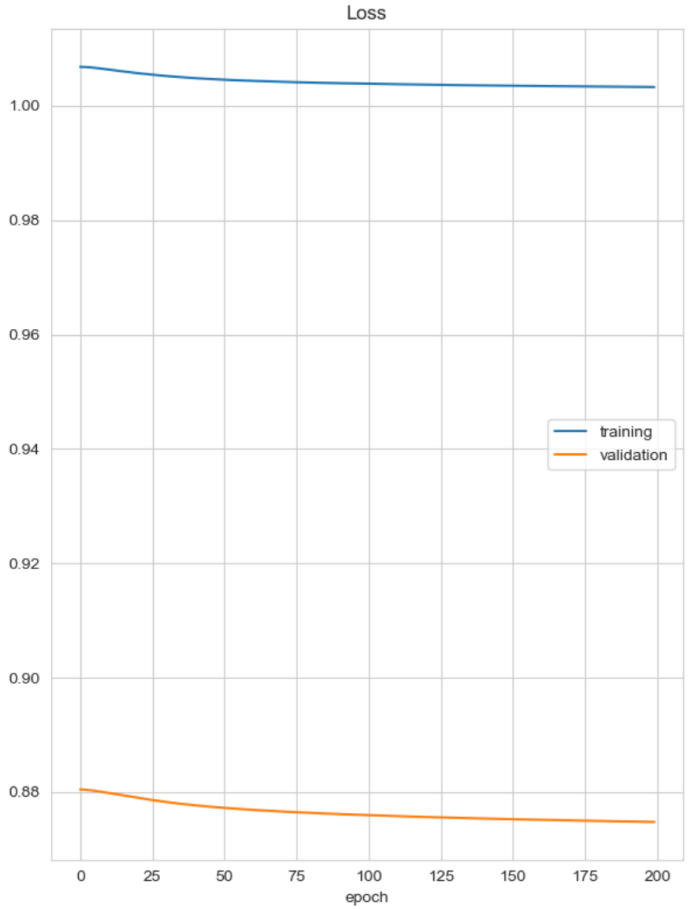

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
MLP_3                                    [624, 3]                  --
├─Linear: 1-1                            [624, 24]                 216
├─ReLU: 1-2                              [624, 24]                 --
├─Linear: 1-3                            [624, 12]                 300
├─ReLU: 1-4                              [624, 12]                 --
├─Linear: 1-5                            [624, 3]                  39
├─Softmax: 1-6                           [624, 3]                  --
==========================================================================================
Total params: 555
Trainable params: 555
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.35
==========================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.19
Params size (MB): 0.00
Estimated Total Size (MB): 0.22
==========================================================================================

MLP_3(
  (hidden1): Linear(in_features=8, out_features=24, bias=True)
  (hidden2): Linear(in_features=24, out_features=12, bias=True)
  (hidden3): Linear(in_features=12, out_features=3, bias=True)
  (act1): ReLU()
  (act2): Softmax(dim=1)
)
```

Multilayer Perceptron – Model 3

Train the model and plot the live training results.






```
accuracy
        training                (min:    0.548, max:    0.550, cur:    0.550)
        validation              (min:    0.674, max:    0.674, cur:    0.674)
Loss
        training                (min:    1.003, max:    1.007, cur:    1.003)
        validation              (min:    0.875, max:    0.880, cur:    0.875)
```

Multilayer Perceptron – Model 3

Evaluate the model.

```
Accuracy: 0.674

success:60 failure:29
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.00	0.00	0.00	16
2	0.68	0.98	0.81	61
accuracy			0.67	89
macro avg	0.23	0.33	0.27	89
weighted avg	0.47	0.67	0.55	89

Apply the model and make a prediction for one case.

```
Predicted: [[0.23223223 0.33779916 0.42996863]] (class=3)
```

Did normalisation improve the results? Why?

Multilayer Perceptron – Model 4

Create another model but change the optimizer to *Adam*.

Model 4:

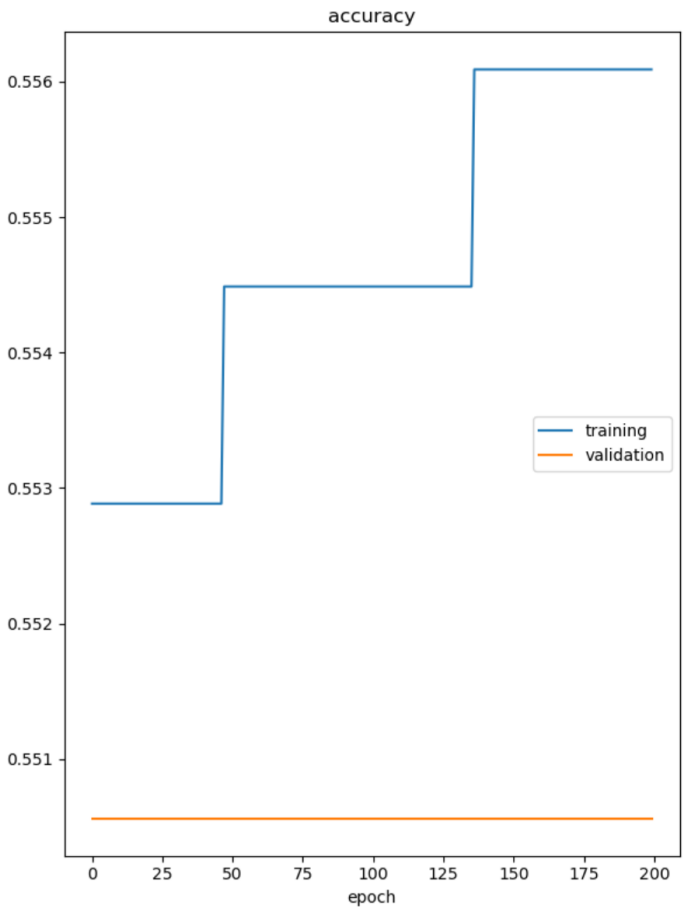
- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss
- **optimiser**: Adam

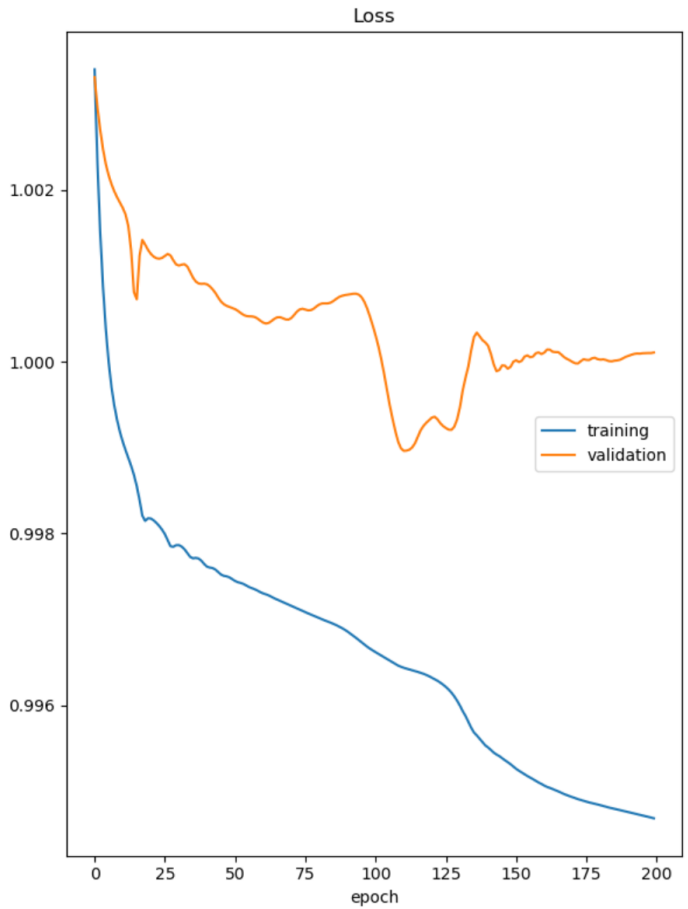
```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
MLP_4                                    [624, 3]                  --
├─Linear: 1-1                            [624, 24]                 216
├─ReLU: 1-2                              [624, 24]                 --
├─Linear: 1-3                            [624, 12]                 300
├─ReLU: 1-4                              [624, 12]                 --
├─Linear: 1-5                            [624, 3]                  39
├─Softmax: 1-6                           [624, 3]                  --
==========================================================================================
Total params: 555
Trainable params: 555
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.35
==========================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.19
Params size (MB): 0.00
Estimated Total Size (MB): 0.22
==========================================================================================

MLP_4(
  (hidden1): Linear(in_features=8, out_features=24, bias=True)
  (hidden2): Linear(in_features=24, out_features=12, bias=True)
  (hidden3): Linear(in_features=12, out_features=3, bias=True)
  (act1): ReLU()
  (act2): Softmax(dim=1)
)
```

Multilayer Perceptron – Model 4

Train the model and plot the live training results.






```
accuracy
        training                (min:    0.553, max:    0.556, cur:    0.556)
        validation              (min:    0.551, max:    0.551, cur:    0.551)
Loss
        training                (min:    0.995, max:    1.003, cur:    0.995)
        validation              (min:    0.999, max:    1.003, cur:    1.000)
```

Multilayer Perceptron – Model 4

Evaluate the model.

```
Accuracy: 0.551

success:49 failure:40
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	22
1	0.00	0.00	0.00	18
2	0.57	1.00	0.73	49
accuracy			0.55	89
macro avg	0.19	0.33	0.24	89
weighted avg	0.31	0.55	0.40	89

Apply the model and make a prediction for one case.

```
Predicted: [[0.09745359 0.0357754  0.866771  ]] (class=3)
```

Did Adam improve the results? Why?

Multilayer Perceptron – Model 5

Create the last model using the best configuration and use an asymmetric loss.

Model 5:

- feedforward network with fully interconnected layers
- **epochs**: 200
- **learning rate**: 0.001
- **layers**: 3 Linear
- **activation fuctions**: ReLU and Softmax
- **loss**: CrossEntropyLoss with class weights
- **optimiser**: Adam

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
MLP_5                                    [624, 3]                  --
├─Linear: 1-1                            [624, 24]                 216
├─ReLU: 1-2                              [624, 24]                 --
├─Linear: 1-3                            [624, 12]                 300
├─ReLU: 1-4                              [624, 12]                 --
├─Linear: 1-5                            [624, 3]                  39
├─Softmax: 1-6                           [624, 3]                  --
==========================================================================================
Total params: 555
Trainable params: 555
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.35
==========================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.19
Params size (MB): 0.00
Estimated Total Size (MB): 0.22
==========================================================================================

MLP_5(
  (hidden1): Linear(in_features=8, out_features=24, bias=True)
  (hidden2): Linear(in_features=24, out_features=12, bias=True)
  (hidden3): Linear(in_features=12, out_features=3, bias=True)
  (act1): ReLU()
  (act2): Softmax(dim=1)
)
```

Multilayer Perceptron – Model 5

Train the model and plot the live training results.

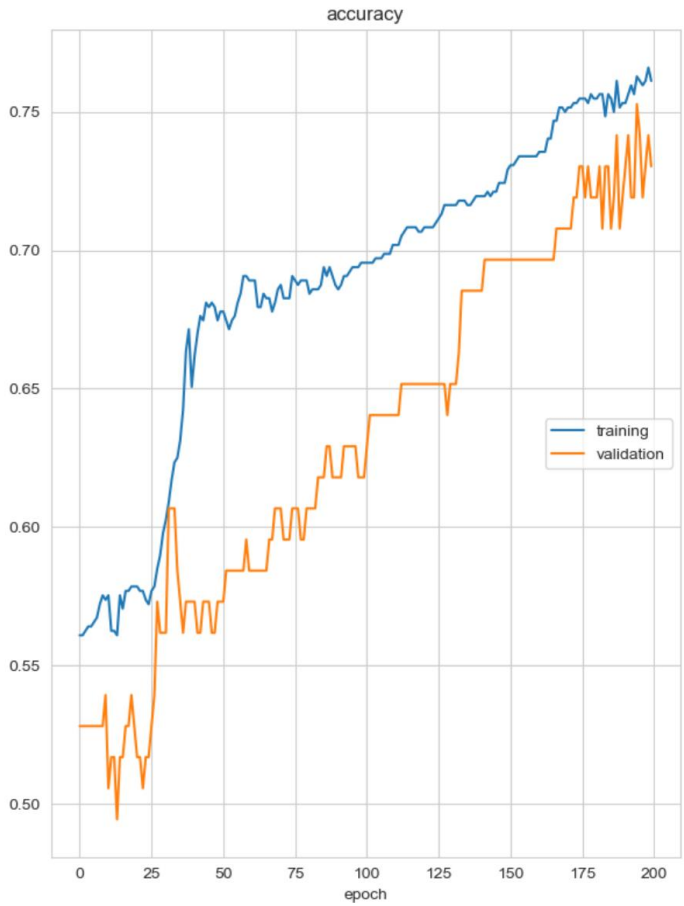
```
def train_model(train_dl, val_dl, model, class_weights):
    liveloss = PlotLosses()

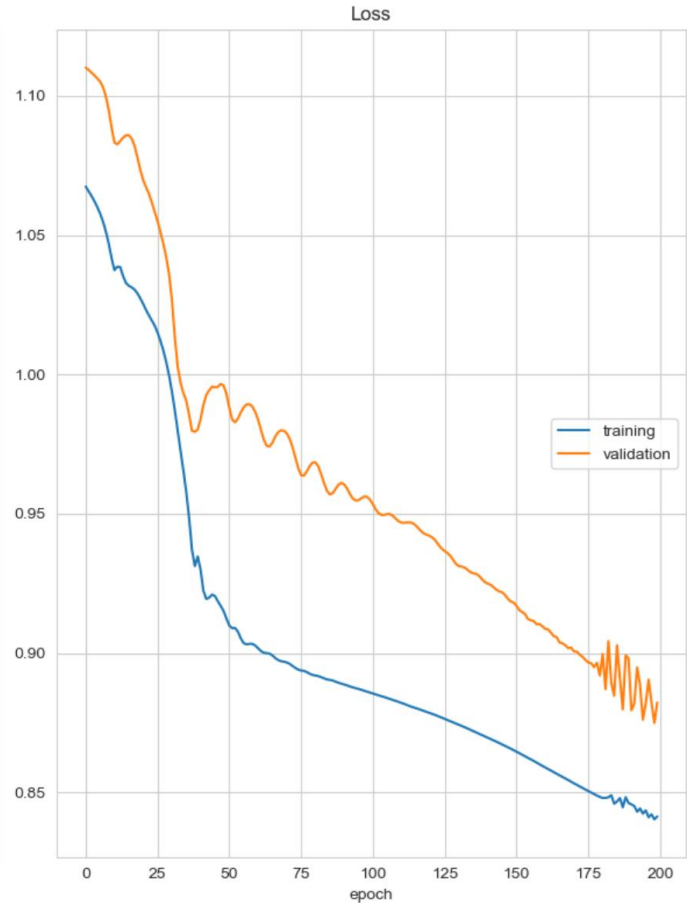
    criterion = CrossEntropyLoss(weight=class_weights)
    optimizer = Adam(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(EPOCHS):
        logs = {}
        model.train()
```

…

```
class_weights = torch.tensor([1.2, 1.2, 0.9])
train_model(train_dl, val_dl, model, class_weights)
```






```
accuracy
        training                (min:    0.561, max:    0.766, cur:    0.761)
        validation              (min:    0.494, max:    0.753, cur:    0.730)
Loss
        training                (min:    0.840, max:    1.067, cur:    0.841)
        validation              (min:    0.875, max:    1.110, cur:    0.882)
```

Multilayer Perceptron – Model 5

Evaluate the model.

```
real:[2] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[1] prediction:[1]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[2] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[3] prediction:[3]
real:[1] prediction:[1]
real:[1] prediction:[1]
```

```
Accuracy: 0.730

success:65 failure:24

              precision    recall  f1-score   support

           0       0.83      0.86      0.84        22
           1       0.00      0.00      0.00        20
           2       0.70      0.98      0.81        47

    accuracy                           0.73        89
   macro avg       0.51      0.61      0.55        89
weighted avg       0.57      0.73      0.64        89

[[19  0  3]
 [ 3  0 17]
 [ 1  0 46]]
```

Apply the model and make a prediction for one case.

```
Predicted: [[0.11224709 0.03453524 0.8532177 ]] (class=3)
```

What conclusions can be draw?

Multilayer Perceptron on Titanic Dataset

Settings: feedforward network with 3 fully interconnected (linear) layers, 200 epochs, LR = 0.001, *ReLU* and *Softmax* as activation functions, *CrossEntropyLoss* as loss

	Initialisation	Normalisation	Loss	Optimiser	Accuracy
Model 1	-	-	CEL	SGD	0.730
Model 2	Kaiming and Xavier	-	CEL	SGD	0.225
Model 3	-	Data Scalling	CEL	SGD	0.674
Model 4	-	-	CEL	Adam	0.551
Model 5	-	-	CEL, weights=[1.2, 1.2, 0.9]	Adam	0.730

default PyTorch initialization was already optimal for this architecture

partial normalization introduces inconsistencies

right weights led to a right balance between compensating class imbalance and maintaining global performance

Adam compensates for initialization and scaling inconsistencies
SGD performs best when the training setup is perfectly balanced

Multilayer Perceptron on Titanic Dataset

Settings: feedforward network with 3 fully interconnected (linear) layers, 200 epochs, LR = 0.001, *ReLU* and *Softmax* as activation functions, *CrossEntropyLoss* as loss

	Initialization	Normalization	Loss	Optimiser	Accuracy
Model 1	-	-	CEL	SGD	0.730
Model 2	Kaiming and Xavier	-	CEL	SGD	0.225
Model 3	-	Data Scaling	CEL	SGD	0.674
Model 4	-	-	CEL	Adam	0.551
Model 5	-	-	CEL, weights=[1.2, 1.2, 0.9]	Adam	0.730

default PyTorch initialization was already optimal for this architecture

partial normalization introduces inconsistencies

right weights led to a right balance between compensating class imbalance and maintaining global performance

Adam compensates for initialization and scaling inconsistencies
SGD performs best when the training setup is perfectly balanced

Multilayer Perceptron on Titanic Dataset

Settings: feedforward network with 3 fully interconnected (linear) layers, 200 epochs, LR = 0.001, *ReLU* and *Softmax* as activation functions, *CrossEntropyLoss* as loss

	Initialization	Normalization	Loss	Optimiser	Accuracy
Model 1	-	-	CEL	SGD	0.730
Model 2	Kaiming and Xavier	-	CEL	SGD	0.225 ↓
Model 3	-	Data Scaling	CEL	SGD	0.674 ↓
Model 4	-	-	CEL	Adam	0.551 ↓
Model 5	-	-	CEL, weights=[1.2, 1.2, 0.9]	Adam	0.730

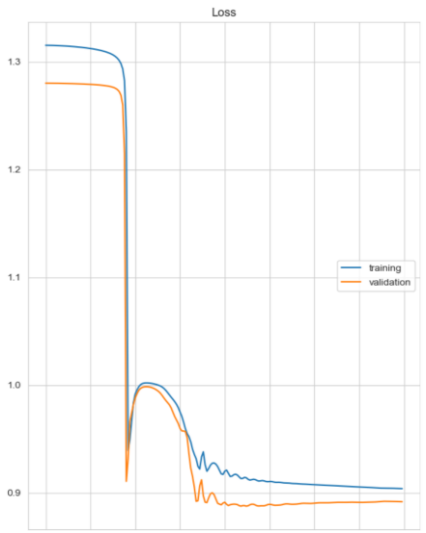
Which model performed better?

What settings can be modified to improve the results?

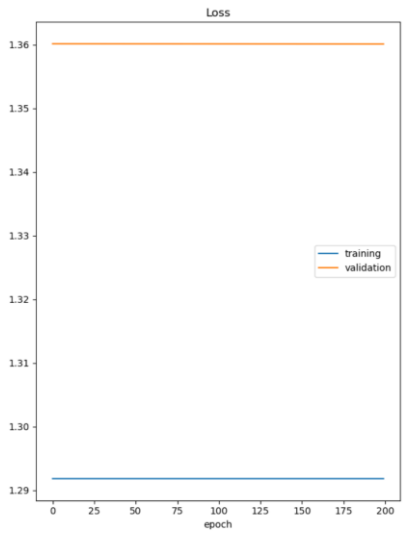
What about the live training plots?

Multilayer Perceptron on Titanic Dataset

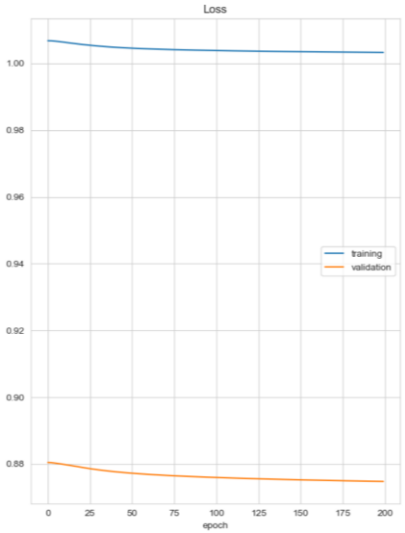
What about the live training plots?



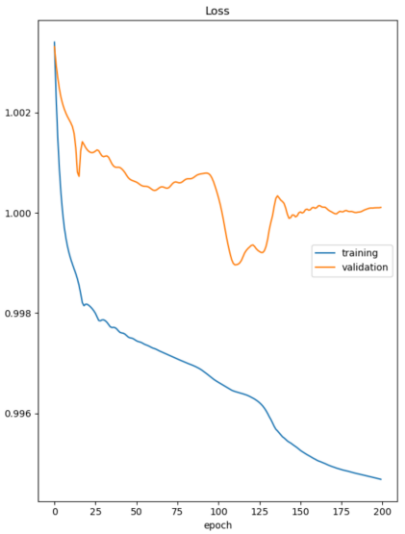
Model 1



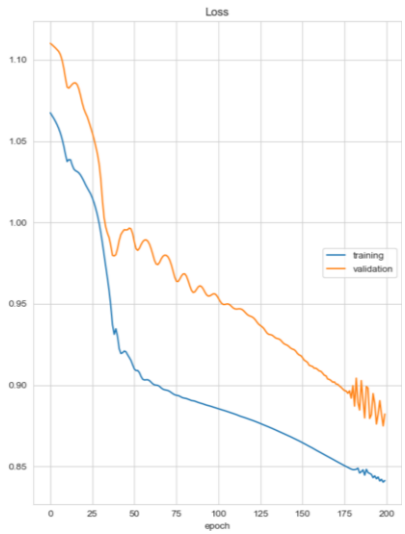
Model 2



Model 3



Model 4

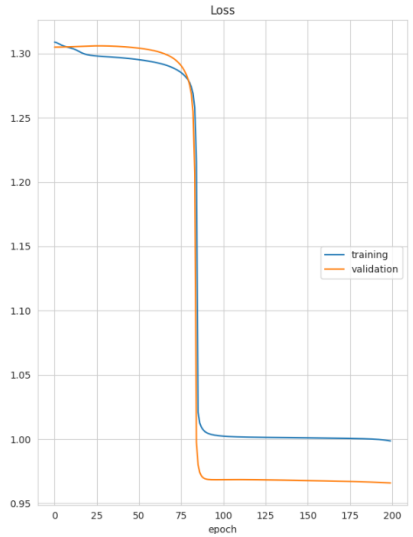


Model 5

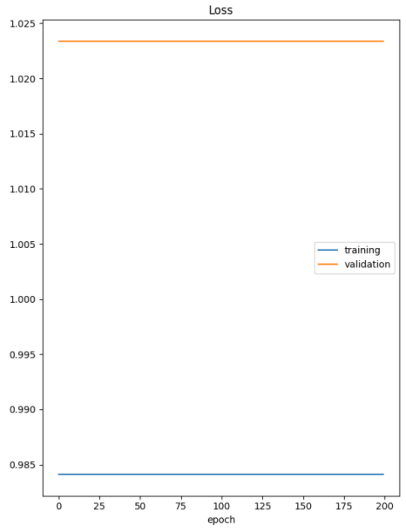
		Model 1	Model 2	Model 3	Model 4	Model 5
accuracy						
	training	0.652)	0.260)	0.550)	0.556)	0.761)
	validation	0.652)	0.191)	0.674)	0.551)	0.730)
Loss						
	training	0.904)	1.292)	1.003)	0.995)	0.841)
	validation	0.892)	1.360)	0.875)	1.000)	0.882)

Multilayer Perceptron on Titanic Dataset

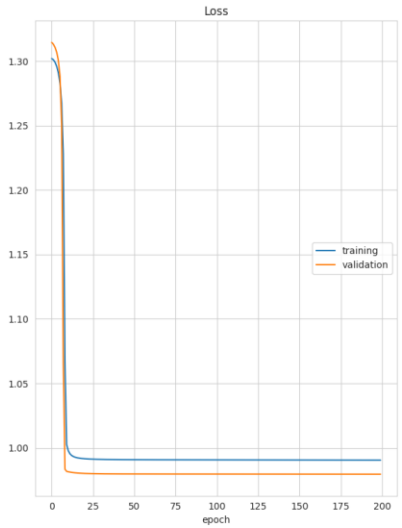
Other runs



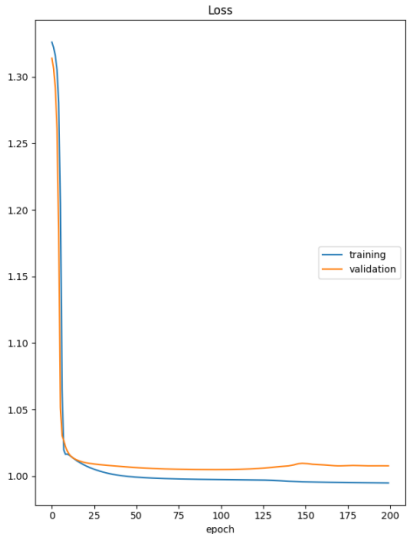
Model 1



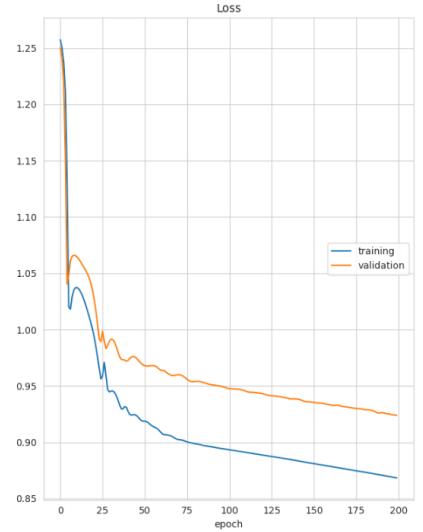
Model 2



Model 3



Model 4



Model 5

		Model 1	Model 2	Model 3	Model 4	Model 5
accuracy						
	training	0.554)	0.567)	0.564)	0.556)	0.716)
	validation	0.584)	0.528)	0.573)	0.551)	0.652)
Loss						
	training	0.999)	0.984)	0.991)	0.995)	0.868)
	validation	0.966)	1.023)	0.980)	1.008)	0.924)

Hands On